

Otomachi Una C++ 模板库

使用手册

Junlin Ye

2026-09-26

这份手册面向首次使用此模板库的选手。先读“接入模板”，再按目录找到所需算法；每个模板都给出真实文件路径、调用顺序、最小示例和使用边界。

示例中的断言同时给出了预期结果。它们用于说明接口，不代替题目的输入输出代码。basic/template.cpp 提供头文件和 main；其他算法文件保持不带头文件和 main 的形式。

本次核对范围是主目录下的全部 C++ 模板；manual 目录中的历史题解、测试程序和旧代码汇编不属于选型范围。

阅读顺序：接入模板 → 选择对应文件 → 按依赖顺序粘贴 → 运行示例 → 替换为题目逻辑。

目录

1 接入模板	4
2 基础 / basic	5
2.1 basic/coordinate-compression.cpp	5
2.2 basic/fast-io.cpp	6
2.3 basic/template.cpp	7
3 数据结构 / data-structure	8
3.1 data-structure/disjoint-set-rollback.cpp	8
3.2 data-structure/disjoint-set.cpp	8
3.3 data-structure/fast-set.cpp	9
3.4 data-structure/fenwick/fenwick.cpp	9
3.5 data-structure/fenwick/max.cpp	10
3.6 data-structure/fenwick/min.cpp	10
3.7 data-structure/fenwick/sum.cpp	11
3.8 data-structure/fhq-treap.cpp	11
3.9 data-structure/li-chao-tree.cpp	12
3.10 data-structure/linear-rmq.cpp	12
3.11 data-structure/ordered-disjoint-interval-tree-fast.cpp	13
3.12 data-structure/ordered-disjoint-interval-tree.cpp	13
3.13 data-structure/priority-queue-two-stacks.cpp	14
3.14 data-structure/rollback-int.cpp	14
3.15 data-structure/rollback-int64.cpp	15
3.16 data-structure/segment-tree/dynamic.cpp	15
3.17 data-structure/segment-tree/iterative-prefix.cpp	16
3.18 data-structure/segment-tree/lazy.cpp	16
3.19 data-structure/segment-tree/merge.cpp	17
3.20 data-structure/segment-tree/point.cpp	18
3.21 data-structure/segment-tree/range-add-max.cpp	19
3.22 data-structure/segment-tree/range-add-min.cpp	19
3.23 data-structure/segment-tree/range-add-sum.cpp	20
3.24 data-structure/sparse-table.cpp	20
4 图论 / graph	21
4.1 graph/block-cut-tree.cpp	21
4.2 graph/chordal.cpp	22
4.3 graph/dijkstra.cpp	23
4.4 graph/directed-mst.cpp	24
4.5 graph/edge-biconnected.cpp	25

4.6	graph/flow/max-flow-hlpp.cpp	25
4.7	graph/flow/min-cost-flow-isap.cpp	26
4.8	graph/flow/min-cost-flow-simplex.cpp	27
4.9	graph/matching/bipartite.cpp	28
4.10	graph/matching/general.cpp	29
4.11	graph/matching/regular-bipartite.cpp	30
4.12	graph/steiner-tree.cpp	31
4.13	graph/two-sat.cpp	31
5	树 / tree	32
5.1	tree/cartesian-tree.cpp	32
5.2	tree/centroid-decomposition.cpp	32
5.3	tree/divide-combine-tree.cpp	33
5.4	tree/heavy-light-decomposition.cpp	34
6	字符串 / string	35
6.1	string/aho-corasick.cpp	35
6.2	string/generalized-suffix-automaton.cpp	36
6.3	string/kmp-z.cpp	36
6.4	string/lyndon.cpp	37
6.5	string/manacher.cpp	38
6.6	string/palindromic-automaton.cpp	39
6.7	string/rolling-hash.cpp	40
6.8	string/runs.cpp	40
6.9	string/suffix-array.cpp	41
6.10	string/suffix-automaton.cpp	42
7	数学 / math	43
7.1	math/big-int.cpp	43
7.2	math/dynamic-mod-int.cpp	43
7.3	math/gaussian-elimination.cpp	44
7.4	math/linear-basis/field-prefix.cpp	45
7.5	math/linear-basis/field.cpp	45
7.6	math/linear-basis/xor-prefix.cpp	46
7.7	math/linear-basis/xor.cpp	46
7.8	math/mod-int.cpp	47
7.9	math/rational.cpp	47
7.10	math/xor-equations.cpp	48
8	数论 / number-theory	49
8.1	number-theory/dujiao-sieve.cpp	49

8.2	number-theory/floor-sum.cpp	49
8.3	number-theory/linear-sieve.cpp	50
8.4	number-theory/modular-arithmetic.cpp	50
8.5	number-theory/pollard-rho.cpp	51
8.6	number-theory/stern-brocot.cpp	52
9	多项式 / polynomial	53
9.1	polynomial/fwt.cpp	53
9.2	polynomial/integer.cpp	54
9.3	polynomial/mtt.cpp	55
9.4	polynomial/ntt-fast.cpp	56
9.5	polynomial/ntt-int64.cpp	57
9.6	polynomial/ntt.cpp	58
10	计算几何 / geometry	59
10.1	geometry/geometry-2d.cpp	59

1 接入模板

一份可以提交的程序怎样组成

编译标准使用 GNU C++17 或更新版本。以 basic/template.cpp 为底稿，在其 main 前按顺序粘贴依赖和算法文件，再填写 main 中的题目逻辑。不要把本库所有文件一起粘贴。

```
#include<bits/stdc++.h>
// Common definitions from basic/template.cpp.
// Paste dependencies and the selected template here.
int main(){
    // Keep the LOCAL file IO block from src here.
    ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);
    // Read input, initialize, build, then query.
}
```

各节“最小示例”放在上面的 main 内即可运行。示例使用固定数据与 assert 检查预期值；提交时改成题目输入，并自行循环输出答案。不要额外复制第二份 main。

调试：编译时加 -DDEBUG 开启 debug/debugl；不加时这两个宏不执行参数表达式。cerr 也仅在定义 DEBUG 时输出；关闭时流表达式不求值。assert 检查模板前置条件，加 -DNDEBUG 会关闭它，初次接入建议保留。

下标：多数图、树、字符串接口从 1 开始；多项式、矩阵、zkw 线段树从 0 开始。区间若无特别说明为闭区间。字符串中的前导空格只是下标占位，不属于实际内容。

多测：清空对象统一使用 set 或 setN；build 用于建表或计算。init、init_mu、init_phi 等保留为预处理接口，是否重算见各节。内部数组为 vector 不代表多个对象之间所有全局表都独立。

先解决依赖和重名

- 可撤销并查集：先粘贴一种 roll back saver，再粘贴 DSU (with undo)。FastODT：先 FastSet，再 FastODT。析合树：先 ST table (linear)，再 xihe_tree。SA/LCP/LCS：先线性 RMQ，再后缀数组；Runs 在这两者之后。
- Gauss<mint>：先选一份模整数定义；Gauss<long double> 不需要模整数。多项式文件已自带模整数、阶乘表，只选一份多项式实现。
- 两种旧线段树同名 segt；HLD 与 DC 都叫 Tree；部分文件自带全局 S、T、rnd、init、inv 等。局部对象不能解决全局重复定义，需要删去重复实现或统一改名。
- DC 和几何的调试成员统一名为 output，可直接与 src 的 debug 宏一起使用。
- 几何需要 using ld=long double;; FastIO 使用 fread/fwrite，支持 Windows 的 GNU C++，无需 POSIX 头文件。

怎样判断示例覆盖了什么

示例检查初始化、参数和返回值的用法，不能证明算法能处理任意题目的数据。点分治和可改 info 的结构需要加入题目逻辑，析合树返回结构后在外部处理答案；手册会明确标出。浮点误差、整数溢出、递归栈深度和内存上限仍由题目约束决定。

示例用于说明接口；本次检查覆盖所有模板编译，以及修改过的字符串查询和 IO 的运行验证。PDF 中不会嵌入大段算法源码；需要复制算法时，以标题中的源码路径为准。

2 基础 / basic

2.1 basic/coordinate-compression.cpp

离散化并原地回写

什么时候用

先收集所有会用到的值，再排序去重，把已登记变量改写成 1..k 的排名。时间 $O(n \log n)$ 。

调用顺序与接口

- `compress<T> c; c.set()` 清空。`c.pb(x)` 要求可修改左值，会保存 `x` 的地址。
- `c.build()` 原地改写 `x`；`c.id(original)` 找已登记原值的排名。
- `c[i]` 由排名恢复原值；`range(l,r)` 返回落在原值闭区间中的排名范围，左端大于右端表示空。

最小示例（放入 `main`）：

```
compress<int> c; c.set();
int a=30,b=10,d=30;
c.pb(a); c.pb(b); c.pb(d); c.build();
assert(a==2 && b==1 && d==2 && c[2]==30);
assert(c.id(30)==2);
```

使用边界

`pb` 后、`build` 前变量必须存活且地址稳定：不要登记即将离开作用域的局部变量，也不要让所属 `vector` 扩容。`id` 参数是原值，不是已改写后的排名。`range` 当前端点参数为 `int`。

2.2 basic/fast-io.cpp

缓冲 IO 与 128 位整数 IO 什么时候用

fread/fwrite 缓冲加速，适用于 Windows 和 Linux 下的 GNU C++17。整数、字符、字符串使用 input/output；cin/cout 也支持有符号及无符号 128 位整数。

调用顺序与接口

- freopen 之后调用 `input.set(stdin); output.set(stdout);`，默认缓冲区 64 KiB，set 的第二个参数可指定大小。
- `input >> x`、`output << x`；`input.read(x)` 返回是否成功，或用 `if(input >> x)` 判断 EOF。
- `char` 读取下一个非空白字符，`string` 读取下一个单词。整数输入必须合法且能存入目标类型；不支持浮点。
- `output.flush()` 刷新用户缓冲区。交互时随后调用 `fflush(stdout)` 刷新 C 标准库缓冲区。

最小示例（放入 main）：

```
// Input: 2 10 20
input.set(stdin); output.set(stdout);
int n; input >> n; lll sum=0;
for(int i=0;i<n;i++){lll x; input >> x; sum+=x;}
output << sum << '\n'; output.flush(); // 30
// Alternative: ordinary streams, without mixing the same FILE.
lll v; istringstream in("-170141183460469231731687303715884105728");
in >> v; ostringstream out; out << v;
assert(out.str()=="-170141183460469231731687303715884105728");
```

使用边界

不要在同一输入流上混用 input 和 cin，或在同一输出流上混用 output 和 cout。文件必须在 Writer 刷新前保持打开；重定向输出前先 flush。流式 128 位输入遇到非法或越界值设置 failbit；快速整数输入以合法数据为前提。原生 MSVC 不支持本模板使用的 `__int128`。

2.3 basic/template.cpp

公共底稿

什么时候用

提供类型别名、调试宏和其他模板依赖的辅助函数。自带标准头文件和 `main`，可直接作为提交程序的底稿。

调用顺序与接口

- 直接复制本文件，在 `main` 前粘贴算法，并在其中填写输入输出与求解逻辑。
- `debug(...)` / `debugl(tag,...)` 接受 1 至 10 个参数；编译时加 `-DDEBUG` 才输出，关闭时参数不求值。
- `chkmin/chkmax` 更新最值并返回是否修改。`first_bit/last_bit` 返回最低/最高置位的位置，参数必须非零。
- `inf<int>`、`inf<ll>` 已特化；其他类型默认值为零，不能当无穷大。

最小示例（放入 `main`）：

```
int a=7; chkmin(a,3);
assert(a==3 && first_bit(12)==2 && last_bit(12)==3);
debug(a);
cerr << "diagnostic\n";
```

使用边界

定义 `LOCAL` 时从 `Otomachi_Una.in` 读入并写入 `Otomachi_Una.out`，打开失败返回 1；否则使用标准输入输出。`DEBUG` 独立控制调试输出。未定义时通过宏屏蔽裸写的 `cerr`，不要写 `std::cerr`。向量输入前必须先分配非空长度；向量输出带方括号并截断长向量，正式答案应自己循环输出。

3 数据结构 / data-structure

3.1 data-structure/disjoint-set-rollback.cpp

可撤销并查集

什么时候用

适合线段树分治或搜索中加入连边后回退。按大小合并、不做路径压缩，单次查找 $O(\log n)$ 。

调用顺序与接口

- 先粘贴一种 saver，再粘贴本文件。
- `unionfind d; d.setN(n)` 初始化；`find/merge` 与普通版类似。
- 通过全局 `version/roll_back` 撤销 `merge`，而不是调用对象成员。

额外依赖（按此顺序粘贴）：`data-structure/rollback-int.cpp`。

最小示例（放入 `main`）：

```
::set(); unionfind d; d.setN(3);
auto t=version(); d.merge(1,2);
assert(d.find(1)==d.find(2));
roll_back(t); assert(d.find(1)!=d.find(2));
```

使用边界

`setN` 前先清空旧历史，不能在旧历史仍引用数组时重新分配。不要自行加入路径压缩。

3.2 data-structure/disjoint-set.cpp

普通并查集

什么时候用

维护集合连通关系，路径压缩配合按大小合并。操作均摊近似 $O(1)$ ，空间 $O(n)$ 。

调用顺序与接口

- `DSU d; d.setN(n)` 创建 $1..n$ 的独立集合。
- `find(x)` 返回代表元；`same(x,y)` 判断同集合。
- `merge(x,y)` 成功返回新代表元，已经连通返回 0。`siz[find(x)]` 为集合大小。

最小示例（放入 `main`）：

```
DSU d; d.setN(4);
d.merge(1,2); assert(d.same(1,2));
assert(d.siz[d.find(1)]==2 && !d.same(1,3));
```

使用边界

再次 `setN` 会清空关系。普通版不能回滚；需要撤销时选择 `unionfind`。

3.3 data-structure/fast-set.cpp

有界整数集合

什么时候用

用分层位集维护集合成员及前驱后继。修改和前驱后继 $O(\log_{64} V)$ ，成员判断 $O(1)$ 。

调用顺序与接口

- `FastSet s; s.setN(v)` 允许存储 $0..v$ ，包含 v 。
- `insert/erase` 幂等，`s[x]` 判断是否存在。
- `prev(x)` 为最大 $\leq x$ 的元素，`next(x)` 为最小 $\geq x$ 的元素；没有则返回 `-1`。
- `front/back` 返回最小/最大元素；`to_vector()` 扫描整个值域输出元素。

最小示例（放入 main）：

```
FastSet s; s.setN(100);
s.insert(3); s.insert(100);
assert(s.next(3)==3 && s.prev(99)==3);
assert(s.next(101)==-1);
```

使用边界

值域太大时不适合使用，空间跟 V 相关而非当前元素个数。需要 `src` 的位运算和最值辅助函数。

3.4 data-structure/fenwick/fenwick.cpp

自定义下标的树状数组

什么时候用

同时提供单点加的 BIT 和区间加的双 BIT。初始化 $O(n)$ ，单次操作 $O(\log n)$ ，空间 $O(n)$ 。

调用顺序与接口

两种结构都用 `setRange(L,R)` 设置闭区间，`setN(n)` 等价于 `setRange(1,n)`。BIT 的 `add(i,v)` 单点加，`ask(r)` 返回 $[L,r]$ 的和，`ask(l,r)` 返回区间和。rars 的 `add(l,r,v)` 区间加，`ask(l,r)` 查区间和。

最小示例（放入 main）：

```
BIT b; b.setRange(-2,2); b.add(0,7);
assert(b.ask(0)==7 && b.ask(-1,1)==7);
rars t; t.setRange(-2,2); t.add(-1,1,3);
assert(t.ask(-2,0)==6);
b.setN(3); assert(b.ask(3)==0);
```

使用边界

下标 0 或负数只要在设定区间内就合法；内部会转成正下标。`ask(L-1)` 返回零，`add(R+1,v)` 是差分哨兵，不计入查询。极端坐标的 $L-1/R+1$ 用 `ll` 计算。rars 的加权差分使用相对位置，不依赖坐标绝对值；值、加权积与答案须在 `ll` 范围内。`set()` 清空；重新 `setRange/setN` 丢弃旧值。文件末尾保留全局 rars 对象 `T`。

3.5 data-structure/fenwick/max.cpp

树状数组：最大值特化

什么时候用

初始化 $O(n)$ ，单点修改和前缀查询 $O(\log n)$ ，空间 $O(n)$ 。

调用顺序与接口

`Max_BIT t; t.setRange(L,R); setN(n)` 调用 `setRange(1,n)`。`upd(i,v)` 把第 i 项取最大值，并非赋值；`ask(r)` 查询 $[L,r]$ 的最大值。

最小示例（放入 `main`）：

```
Max_BIT t; t.setRange(-2,2); t.upd(0,5); t.upd(0,3);
assert(t.ask(1)==5);
```

使用边界

初值和空前缀返回 `LLONG_MIN`。只支持单点 `chmax`；不能把已有大值减小。没有 `ask(l,r)`，最值不能像和一样用两个前缀相减；任意点赋值或一般区间最值请用线段树。三个类名不同，可组合粘贴；`set()` 清空。

3.6 data-structure/fenwick/min.cpp

树状数组：最小值特化

什么时候用

初始化 $O(n)$ ，单点修改和前缀查询 $O(\log n)$ ，空间 $O(n)$ 。

调用顺序与接口

`Min_BIT t; t.setRange(L,R); setN(n)` 调用 `setRange(1,n)`。`upd(i,v)` 把第 i 项取最小值，并非赋值；`ask(r)` 查询 $[L,r]$ 的最小值。

最小示例（放入 `main`）：

```
Min_BIT t; t.setRange(-2,2); t.upd(0,5); t.upd(0,3);
assert(t.ask(1)==3);
```

使用边界

初值和空前缀返回 `LLONG_MAX`。只支持单点 `chmin`；不能把已有小值增大。没有 `ask(l,r)`，最值不能像和一样用两个前缀相减；任意点赋值或一般区间最值请用线段树。三个类名不同，可组合粘贴；`set()` 清空。

3.7 data-structure/fenwick/sum.cpp

树状数组：和特化

什么时候用

初始化 $O(n)$ ，单点修改和前缀查询 $O(\log n)$ ，空间 $O(n)$ 。

调用顺序与接口

`Sum_BIT t; t.setRange(L,R); setN(n)` 调用 `setRange(1,n)`。`add(i,v)` 单点加；`ask(r)` 查询 $[L,r]$ 的和；`ask(l,r)` 查询区间和。

最小示例（放入 `main`）：

```
Sum_BIT t; t.setRange(-2,2); t.add(0,5); t.add(1,-3);
assert(t.ask(0)==5 && t.ask(0,1)==2);
```

使用边界

初始值全零；允许 `add(R+1,v)` 差分哨兵及 `ask(L-1)` 空前缀。支持负坐标，查询不会修改树。三个类名不同，可组合粘贴；`set()` 清空。

3.8 data-structure/fhq-treap.cpp

按键或按序列位置分裂的 `Treap`

什么时候用

随机平衡二叉树。外部维护根编号 `rt`，空树为 0，普通 `split/merge` 期望 $O(\log n)$ 。

调用顺序与接口

- `FHQ t; t.set()` 清空；`newnode(x)` 建一个 `key=x` 的节点。
- `split_by_key(rt,k,a,b)` 分为 `key<=k` 与 `key>k`；`split_by_size(rt,k,a,b)` 分出前 `k` 个。
- `rt=merge(a,b)` 要求 `a` 所有元素应在 `b` 前面；按键维护时要求 `max(a)<=min(b)`。
- `merge_it(a,b)` 可合并键域交错的两树。`push(rt,v)` 给整个非空子树的 `key` 加 `v`。

最小示例（放入 `main`）：

```
FHQ t; t.set(); int rt=0;
for(int v:{2,5,8})rt=t.merge(rt,t.newnode(v));
int a,b; t.split_by_key(rt,5,a,b);
assert(t.siz[a]==2 && t.siz[b]==1);
rt=t.merge(a,b); assert(t.siz[rt]==3);
```

使用边界

`setN(n)` 只是创建 `n` 个孤立节点，不会把它们连成树。按键分裂依赖排序不变量；区间加可能破坏该不变量。没有现成的区间翻转或持久化。全局 `rnd` 可能与别的模板重名。

3.9 data-structure/li-chao-tree.cpp

线段李超树

什么时候用

插入只在一段整数横坐标上有效的直线，查询某点的最小值。

调用顺序与接口

- lichao t; t.set() 清空。
- add(line{k,b},l,r) 插入 $y=kx+b$ ；省略 l,r 表示整个默认值域。
- ask(x) 返回最小纵坐标，没有覆盖返回 $2e18$ 。

最小示例（放入 main）：

```
lichao t; t.set();
t.add({2,3},0,10); t.add({-1,8},0,10);
assert(t.ask(2)==6);
```

使用边界

横坐标及斜率限制在 $[-10^9, 10^9]$ ，有效端点的纵坐标需在 $[-2e18, 2e18]$ 。整条线插入 $O(\log V)$ ，线段插入 $O(\log^2 V)$ ，查询 $O(\log V)$ 。

3.10 data-structure/linear-rmq.cpp

四毛子 RMQ：线性预处理与空间

什么时候用

静态数组最值。采用块内单调栈位掩码、块间 ST 表；在机器字位运算为 $O(1)$ 的模型下，预处理及空间 $O(n)$ ，查询 $O(1)$ 。

调用顺序与接口

Linear_RMQ<int> t; t.build(a) 接受从 0 开始的 vector，内部保存副本。ask(l,r) 查询闭区间最小值，pos(l,r) 返回最小值下标；相等取最左。Linear_RMQ<int,true> 改为最大值，其他接口相同。

最小示例（放入 main）：

```
vector<int> a{4,1,1,3};
Linear_RMQ<int> mn; mn.build(a);
assert(mn.ask(0,3)==1 && mn.pos(0,3)==1);
Linear_RMQ<int,true> mx; mx.build(a);
assert(mx.ask(1,3)==3);
```

使用边界

块长 $B=\max(1, \text{floor}(\log_2 n))$ ，不是固定常数；块数 $m=\text{ceil}(n/B)$ ，块间表 $O(m \log m)=O(n)$ 。每个元素存一份块前缀的单调栈掩码，一次清除低位和 ctz 即可查询块内最左最值。 $n < \text{INT_MAX}$ ，块长最多 30，使用 unsigned 位运算。可 build 空数组，但不能查询；修改数组后须重新 build。set() 清空。不需要普通 ST 表或笛卡尔树作为依赖。

3.11 data-structure/ordered-disjoint-interval-tree-fast.cpp

位集加速的区间覆盖

什么时候用

借助 FastSet 存储区间右端点。下标 1..n，适合值域固定的区间覆盖。

调用顺序与接口

- 先粘贴 FastSet，再粘贴本文件。
- `Fast_ODT t; t.setN(n)` 分配空间；先 `insert(1,n,c0)` 明确初始颜色。
- `insert(l,r,c)` 覆盖区间并返回旧的 `{L,R,col}` 段；`extract(l,r)` 获取当前片段。

额外依赖（按此顺序粘贴）：`data-structure/fast-set.cpp`。

最小示例（放入 `main`）：

```
Fast_ODT t; t.setN(5); t.insert(1,5,0);
t.insert(2,4,7); auto a=t.extract(2,3);
assert(a.size()==1 && a[0][2]==7);
```

使用边界

`extract` 会分裂边界。不保证任意操作序列的高效性，成本仍取决于访问的段数。

3.12 data-structure/ordered-disjoint-interval-tree.cpp

有序区间集合

什么时候用

维护若干不相交的常值闭区间。适合区间覆盖以及依赖当前分段的统计。

调用顺序与接口

- `ODT t; t.set()` 清空；`add(l,r,c)` 只插入一个区间，不会自动清除旧区间。
- 赋值应先 `erase(l,r)` 再 `add(l,r,c)`；`erase` 返回被覆盖的旧段。
- `extract(l,r)` 返回当前区间片段，每段字段为 `l,r,c`。

最小示例（放入 `main`）：

```
ODT t; t.set(); t.add(1,5,0);
t.erase(2,4); t.add(2,4,7);
auto a=t.extract(2,3);
assert(a.size()==1 && a[0].c==7);
```

使用边界

`extract` 会切开边界，可能增加内部段数；没有自动合并相邻同色段。一次操作可能遍历很多段，不保证对任意输入都是对数复杂度。

3.13 data-structure/priority-queue-two-stacks.cpp

把优先队列删除转为栈撤销

什么时候用

适合只支持栈式加入/撤销，却需要删除最大权元素的离线或可撤销结构。模板只安排顺序，实际状态由调用方维护。

调用顺序与接口

- `pq2stack<T> q; q.set()` 初始化；`push(x,w)` 登记元素及删除优先级。
- 每次查询前先 `prep()`，按返回顺序把元素加入外部状态。
- `pop()` 删除最大权元素，返回外部栈需要撤销的次数；之后再次 `prep` 会给出重新加入的元素。

最小示例（放入 `main`）：

```
pq2stack<int> q; q.set(); vector<int> st;
q.push(10,1); q.push(20,2);
for(int x:q.prep())st.push_back(x);
int k=q.pop(); while(k--)st.pop_back();
for(int x:q.prep())st.push_back(x);
assert(st.size()==1 && st[0]==10);
```

使用边界

不能把 `pop` 的返回值当成被删元素。每次重加入都需要重新产生撤销记录；同权元素删除顺序不固定。单次可能重排大量元素，收益依赖整段操作序列。

3.14 data-structure/rollback-int.cpp

记录并撤销变量修改

什么时候用

保存变量的地址和旧值，用版本号撤销一段修改。每次保存和撤销一次记录都是 $O(1)$ 。

调用顺序与接口

- 修改前先 `save(x)`。
- `version()` 返回当前记录数，`roll_back(t)` 逆序恢复到该记录数。
- `set()` 仅丢弃历史，不恢复变量。

最小示例（放入 `main`）：

```
::set(); int x=7;
auto t=version(); save(x); x=20;
roll_back(t); assert(x==7);
```

使用边界

保存期间变量地址必须保持有效，不能让包含它的 `vector` 扩容。每个版本号只能用于当前历史；两种 `saver` 不能同时粘贴。

3.15 data-structure/rollback-int64.cpp

记录并撤销变量修改

什么时候用

保存变量的地址和旧值，用版本号撤销一段修改。每次保存和撤销一次记录都是 $O(1)$ 。

调用顺序与接口

- 修改前先 `save(x)`。
- `version()` 返回当前记录数，`roll_back(t)` 逆序恢复到该记录数。
- `set()` 仅丢弃历史，不恢复变量。

最小示例（放入 `main`）：

```
::set(); ll x=7;
auto t=version(); save(x); x=20;
roll_back(t); assert(x==7);
```

使用边界

保存期间变量地址必须保持有效，不能让包含它的 `vector` 扩容。每个版本号只能用于当前历史；两种 `saver` 不能同时粘贴。

3.16 data-structure/segment-tree/dynamic.cpp

大值域稀疏单点统计

什么时候用

在整数键 $-10^9..10^9$ 上做单点加、区间和；节点带键交换以压缩空间。单次 $O(\log V)$ 。

调用顺序与接口

- `dynamic_segt t; t.set()` 清空；`set(m)` 可预留节点。
- `add(x,v)` 给键 `x` 的权值加 `v`；`ask(l,r)` 求键区间权值和。

最小示例（放入 `main`）：

```
dynamic_segt t; t.set();
t.add(-100,3); t.add(200,4);
assert(t.ask(-100,0)==3);
```

使用边界

空键权值为零。与主席树不同：不保留历史版本，也不支持复用旧根。文件有全局对象 `T`。

3.17 data-structure/segment-tree/iterative-prefix.cpp

非递归线段树框架

什么时候用

单点赋值和闭区间聚合，当前 info 维护区间和与最大前缀和（允许空前缀）。单次 $O(\log n)$ 。

调用顺序与接口

- `zkw_segt t; t.setN(n)` 下标为 $0..n-1$ 。
- `upd(i,a,b)` 设置该叶 info 的 $s=a$ 、 $mx=b$ 。
- 合并两段时 $s=L.s+R.s$ ， $mx=\max(L.mx, L.s+R.mx)$ ；ask 返回最终 mx 。

最小示例（放入 main）：

```
zkw_segt t; t.setN(3);
t.upd(0,-4,0); t.upd(1,9,9); t.upd(2,-2,0);
assert(t.ask(0,2)==5);
```

使用边界

普通数组的叶子设为 $s=a[i]$ 、 $mx=\max(0,a[i])$ 。若 s 全为零且 mx 非负，也可用于区间最大值。其他统计应重写 info 的合并和单位元；合并必须满足结合律。下标从 0 开始。

3.18 data-structure/segment-tree/lazy.cpp

通用线段树（带懒标记）

什么时候用

默认维护区间和，下标 $1..n$ ，初值全零。递归部分只处理 info，按题目改写信息与标记的运算。

调用顺序与接口

- `segt t; t.setN(n)` 初始化。
- `upd(i,v)` 单点赋值；`ask(l,r)` 区间和，`ask(i)` 单点值。
- `add(i,v)` 单点加，`add(l,r,v)` 区间加。

最小示例（放入 main）：

```
segt t; t.setN(3); t.upd(2,5);
t.add(1,2,2); assert(t.ask(1,3)==9);
```

使用边界

info + info 按左右顺序合并；upd 在叶子整体替换 info，单点 add 使用 `info(delta,0)`，通过 `info += info` 更新叶子。初值在 build 的 `info(0,1)` 处修改。参数转换、提取答案只改公开接口。USE_COMPARE 默认开启；自定义信息不支持减法与比较时，删除文件顶部的 define 行，对应运算和二分函数就不编译。默认二分要求所有点值非负、目标为正且不超过总和。set/setN 清空重建。两版同名 segt，只选一份。lazt += lazt 的顺序是旧标记后接新标记；info += lazt 负责作用标记，区间长度等元数据由 info 保存。不能仅因支持单点加，就直接认为相同统计能支持区间加。

3.19 data-structure/segment-tree/merge.cpp

可合并的多棵线段树

什么时候用

维护多棵互不共享节点的树，值域 $0..n$ 。当前权值设计用于统计所有键 $y \geq x$ 的 $(y-x) * \text{权值}$ 。

调用顺序与接口

- `segt_merge t; t.setN(n)` 初始化池，外部根变量设为 0。
- `add(rt,y,w)` 给键 y 加权 w ，根按引用更新。
- `ask(rt,x)` 返回 $\sum_{y \geq x} (y-x) * w[y]$ 。
- `merge(a,b)` 把 b 合入 a 并把 b 置零。

最小示例（放入 `main`）：

```
segt_merge t; t.setN(10); int a=0,b=0;
t.add(a,5,2); t.add(b,8,1); t.merge(a,b);
assert(b==0 && t.ask(a,3)==9);
```

使用边界

这是破坏性合并，不是可持久化。合并后不能再使用旧根副本。单点操作 $O(\log V)$ ，合并成本取决于重叠节点数；若想查询普通权值和，要改 `info/query` 的对外接口。

3.20 data-structure/segment-tree/point.cpp

通用线段树（不带懒标记）

什么时候用

默认维护区间和，下标 1..n，初值全零。递归部分只处理 info，按题目改写信息与标记的运算。

调用顺序与接口

- segt t; t.setN(n) 初始化。
- upd(i,v) 单点赋值; ask(l,r) 区间和, ask(i) 单点值。
- add(i,v) 单点加, 不支持区间加。

最小示例（放入 main）:

```
segt t; t.setN(3); t.upd(2,5);
t.add(2,2); assert(t.ask(1,3)==7);
```

使用边界

info + info 按左右顺序合并; upd 在叶子整体替换 info, 单点 add 使用 info(delta,0), 通过 info += info 更新叶子。初值在 build 的 info(0,1) 处修改。参数转换、提取答案只改公开接口。USE_COMPARE 默认开启; 自定义信息不支持减法与比较时, 删除文件顶部的 define 行, 对应运算和二分函数就不编译。默认二分要求所有点值非负、目标为正且不超过总和。set/setN 清空重建。两版同名 segt, 只选一份。

改写示例: 单点加、区间最大子段和（非空）删除顶部的 #define USE_COMPARE, 把 info 替换为下面这段; 区间 ask 的返回字段从 s 改为 mx, 其他部分不改。len=0 的参数表示单点增量, len=1 表示叶子; 合并的左右区间均非空。

```
struct info{
    ll s,pre,suf,mx,len;
    info(ll x=0,ll n=0):s(x),pre(x),suf(x),mx(x),len(n){}
    inline info& operator +=(const info &y){
        if(!y.len){ // point add: y is info(delta,0)
            s+=y.s;pre=suf=mx=s;return *this;
        }
        mx=max({mx,y.mx,suf+y.pre});
        pre=max(pre,s+y.pre);suf=max(y.suf,y.s+suf);
        s+=y.s;len+=y.len;return *this;
    }
    inline info operator +(const info &y)const{return info(*this)+=y;}
};
```

3.21 data-structure/segment-tree/range-add-max.cpp

区间加、区间最大值

什么时候用

默认闭区间下标 1..n、初值全零，初始化 $O(n)$ ，单次修改查询 $O(\log n)$ 。

调用顺序与接口

- `setN(n)` 或 `set(l,r)` 指定有效闭区间。
- `upd(i,v)` 是赋值，`add(i,v)` 是加法；`add(l,r,v)` 区间加。
- `ask(l,r)` 查询，`ask(i)` 查单点。

最小示例（放入 main）：

```
MaxAdd_segt t; t.setN(3);
t.upd(2,5); t.add(1,2,2);
assert(t.ask(1,3)==7);
```

使用边界

这是区间加的特化实现；要更换信息或标记类型，使用外面的两份通用模板。`set/setN` 直接初始化全零节点，不递归建树。若初始数组不是全零，应逐点 `upd`；负数数组不需要改初始化，只要把所有位置正确赋值。三个类名不同，可以组合粘贴。

3.22 data-structure/segment-tree/range-add-min.cpp

区间加、区间最小值

什么时候用

默认闭区间下标 1..n、初值全零，初始化 $O(n)$ ，单次修改查询 $O(\log n)$ 。

调用顺序与接口

- `setN(n)` 或 `set(l,r)` 指定有效闭区间。
- `upd(i,v)` 是赋值，`add(i,v)` 是加法；`add(l,r,v)` 区间加。
- `ask(l,r)` 查询，`ask(i)` 查单点。

最小示例（放入 main）：

```
MinAdd_segt t; t.setN(3);
t.upd(2,5); t.add(1,2,2);
assert(t.ask(1,3)==0);
```

使用边界

这是区间加的特化实现；要更换信息或标记类型，使用外面的两份通用模板。`set/setN` 直接初始化全零节点，不递归建树。若初始数组不是全零，应逐点 `upd`；负数数组不需要改初始化，只要把所有位置正确赋值。三个类名不同，可以组合粘贴。

3.23 data-structure/segment-tree/range-add-sum.cpp

区间加、区间和

什么时候用

默认闭区间下标 1..n、初值全零，初始化 $O(n)$ ，单次修改查询 $O(\log n)$ 。

调用顺序与接口

- `setN(n)` 或 `set(l,r)` 指定有效闭区间。
- `upd(i,v)` 是赋值，`add(i,v)` 是加法；`add(l,r,v)` 区间加。
- `ask(l,r)` 查询，`ask(i)` 查单点。
- `find_first(k)` 从左累计达到 k 的位置；`find_last(k)` 从右累计达到 k 的位置。

最小示例（放入 main）：

```
AddSum_segt t; t.setN(3);
t.upd(2,5); t.add(1,2,2);
assert(t.ask(1,3)==9);
```

使用边界

这是区间加的特化实现；要更换信息或标记类型，使用外面的两份通用模板。`set/setN` 直接初始化全零节点，不递归建树。区间长度由递归参数计算，不存入节点；`ask` 和二分携带祖先标记，不下传、不改树。使用二分查询时所有点值必须非负，且 $1 \leq k \leq$ 总和； k 参数为 `ll`。三个类名不同，可以组合粘贴。

3.24 data-structure/sparse-table.cpp

静态区间最值

什么时候用

数组不修改、需要多次查询闭区间最大值时使用。预处理 $O(n \log n)$ ，查询 $O(1)$ ，空间 $O(n \log n)$ 。

调用顺序与接口

- `ST_table t; t.setN(n)` 分配下标 1..n，要求 $n \geq 1$ 。
- 填入 `t.a[0][i]` 后调用 `t.build()`。
- `t.ask(l,r)` 返回最大值；要维护最小值，先把 `chk` 改成 `min`。

最小示例（放入 main）：

```
ST_table t; t.setN(3);
t.a[0][1]=4; t.a[0][2]=1; t.a[0][3]=8;
t.build(); assert(t.ask(1,2)==4);
```

使用边界

修改原数组后必须重新 `build`。文件末尾已有全局对象 `s`，可以直接用它，也可以另建局部对象。

4 图论 / graph

4.1 graph/block-cut-tree.cpp

点双连通分量与圆方树

什么时候用

无向图的点双分解，支持非连通图和重边，不支持自环。构建时间 $O(n+m)$ 。

调用顺序与接口

- `block_cut g; g.setN(n)` 后 `add_edge`，再 `build`。
- 原点仍为 `1..n`，方点为 `n+1..scc`，`tr` 是圆方森林邻接表。
- 第 `k` 个方点 `n+k` 的原图边存在 `ec[k]` 中。`ff`、`dep` 是森林上的父亲和深度。

最小示例（放入 `main`）：

```
block_cut g; g.setN(3);
g.add_edge(1,2); g.add_edge(2,3); g.build();
assert(g.scc==5 && g.tr[2].size()==2);
```

使用边界

`scc` 是最终最大节点编号，不是分量个数，方点数应为 `scc-n`。割点可以属于多个点双，不能像 SCC 一样给每个原点分配唯一块号。递归深度可能达到 $O(n)$ 。

4.2 graph/chordal.cpp

弦图判定与完美消除序列

什么时候用

判断无向图中是否不存在长度至少为 4 的无弦诱导环。如果是弦图，返回一个完美消除序列。

调用顺序与接口

- Chordal g; g.setN(n), 点下标 1..n; g.add_edge(u,v) 加无向边。自环忽略, 重边在求解时去重。
- bool ok=g.solve()。成功时 peo[0..n-1] 按从前到后给出消除顺序, rk[u] 是 u 在序列中的 1 下标位置。
- 依次消除 peo 中的点时, 每个点尚未消除的邻居构成团。不要把 MCS 的选点顺序直接作为消除顺序, 两者相反。
- 失败时清空 peo、rk。空图和非连通图均支持; 可以继续加边再求解。

最小示例 (放入 main):

```
Chordal g; g.setN(4);
g.add_edge(1,2); g.add_edge(2,3);
g.add_edge(3,4); g.add_edge(4,1);
assert(!g.solve() && g.peo.empty());
g.add_edge(1,3); assert(g.solve());
assert(g.peo.size()==4);
for(int i=0;i<4;i++)assert(g.rk[g.peo[i]]==i+1);
```

使用边界

时间、空间均为 $O(n+m)$, m 为输入边数。MCS 用桶维护已选邻居数; 验证时把结点按最靠前的后继邻居分组, 统一检查后继邻居的包含关系, 避免对每个邻域两两枚举。返回 false 时只给出判定, 不构造无弦环证据。

4.3 graph/dijkstra.cpp

非负权单源最短路

什么时候用

堆优化 Dijkstra，有向图，复杂度 $O((n+m) \log n)$ 。

调用顺序与接口

- `graph_SSSP g; g.setN(n); g.setS(s)` 设置图和源点。
- `add_edge(u,v,w)` 要求 $w \geq 0$ ；无向图要加两个方向。
- `SSSP()` 返回长度 $n+1$ 的距离数组，0 号不用，不可达为 10^{18} 。

最小示例（放入 `main`）：

```
graph_SSSP g; g.setN(3); g.setS(1);
g.add_edge(1,2,4); g.add_edge(2,3,6);
auto d=g.SSSP(); assert(d[3]==10);
```

使用边界

有限最短路必须小于不可达哨兵 10^{18} ，边权相加不能溢出。不支持负权；不会返回具体路径。

4.4 graph/directed-mst.cpp

指定根的最小树形图

什么时候用

在带权有向图中，以 S 为根，选取 $n-1$ 条边，使 S 能沿选边到达每个点，且总权最小。每个非根点恰有一条入边。

调用顺序与接口

- `Directed_MST g; g.setN(n); id=g.add_edge(u,v,w)` 加入 u 指向 v 的边， w 为 ll ，返回从 1 开始的边编号。
- `auto a=g.solve(S)` 返回 `result`。`a.ok=false` 表示不存在以 S 为根的生成树形图，此时 `id` 为空。
- 有解时 `a.cost` 为总权，`a.id[v]` 为 v 的入边编号，`a.id[S]=0`；遍历所有非根点即可得到实际边集，父亲是 `g.e[a.id[v]].u`。
- 点和 `id` 数组均为 1 下标。 $n=1$ 时有解、总权为 0；要求 $1 \leq S \leq n$ 。允许负权、重边、自环；同端点只保留最小权，平权保留较早加入的边。

最小示例（放入 main）：

```
Directed_MST g; g.setN(4);
g.add_edge(1,2,5); g.add_edge(1,3,8);
g.add_edge(2,3,-2); g.add_edge(3,2,-1);
g.add_edge(3,4,1);
auto a=g.solve(1);
assert(a.ok && a.cost==4 && a.id[1]==0);
assert(a.id[2]==1 && a.id[3]==3 && a.id[4]==5);
assert(!g.solve(4).ok);
```

使用边界

设 n 为点数、 m 为输入边数，时间 $O((n+m) \log(n+1))$ ，空间 $O(n+m)$ ；任意显式图至少需要 $O(m)$ 读边，只有稀疏图才能写成 n 乘多对数。使用左偏堆、可回退并查集缩环并恢复选边。中间权值和 `cost` 都是 `__int128`；输出总权时先抄 `basic/fast-io.cpp`，不要直接强转 `ll`。`solve` 不修改原图，可更换根重新求解。

4.5 graph/edge-biconnected.cpp

边双连通分量及桥森林

什么时候用

无向图删除桥后缩点，支持重边和非连通图。时间 $O(n+m)$ 。

调用顺序与接口

- `bi_edge g; g.setN(n)` 后加边。
- `auto e=g.build()` 返回连接不同分量的边表；每端是分量编号。
- `bel[u]` 表示点所属分量，分量编号 `1..scnt`。

最小示例（放入 `main`）：

```
bi_edge g; g.setN(3);
g.add_edge(1,2); g.add_edge(1,2); g.add_edge(2,3);
auto e=g.build();
assert(g.bel[1]==g.bel[2] && g.bel[2]!=g.bel[3]);
assert(e.size()==1);
```

使用边界

桥森林的顶点是分量，不是原图点。与圆方树分别处理边双、点双，不能互换。

4.6 graph/flow/max-flow-hlpp.cpp

HLPP 最大流

什么时候用

有向容量图的最大流。使用 HLPP：优先处理最高标号的活跃点，配合反向 BFS 全局重标号、当前弧和 gap 剪枝。邻接边编号连续存储，整个算法不使用递归。不能到达汇点的多余预流会退回源点，因此返回的边流量满足流量守恒。

调用顺序与接口

- `Flow_Graph g; g.setN(n); g.setST(s,t)` 初始化，点编号 `1..n`，源汇必须不同。
- `id=g.add_edge(u,v,cap)` 只添加一条有向边，自动生成反向残量边。
- `max_flow()` 返回累计最大流值；`edge_flow(id)` 查看返回的正向边上的流量。

最小示例（放入 `main`）：

```
Flow_Graph g; g.setN(3); g.setST(1,3);
int id=g.add_edge(1,2,5); g.add_edge(2,3,3);
assert(g.max_flow()==3 && g.edge_flow(id)==3);
assert(g.max_flow()==3);
```

使用边界

重复求流返回总流量，不是新增流量；添加边后继续增广。`setN` 清空图，`new_node` 保留旧图添加点。无向容量边需要根据建模显式加边。每条边容量和总流量须能用 `ll` 表示，预流余量使用 `__int128`，源点出边容量之和可以超过 `ll`。更换源汇后从原始容量重算；设置相同源汇保留已有流。不同图上不保证比 Dinic 更快。

4.7 graph/flow/min-cost-flow-isap.cpp

最小费用最大流：Dijkstra + ISAP 备份

什么时候用

作为网络单纯形版的替代实现。初始 SPFA 求势能，每轮 Dijkstra 求最短路，再在零约化费用子图上用非递归 ISAP 批量增广，包含反向 BFS、当前弧和 gap 剪枝。两份模板均使用 Cost_Flow_Graph，接口相同，选一份粘贴即可。

调用顺序与接口

- `setN(n)`; `setST(s,t)` 初始化，点编号 1..n; `set()` 清空。
- `add_edge(u,v,cap,cost)` 添加有向边并返回正向边编号。
- `min_cost_flow()` 返回总流量、总费用; `edge_flow(id)` 查询边流量。
- `new_node()` 保留已有边并新增一个点。编辑图后再次求解会从原始容量重算。

最小示例（放入 main）:

```
Cost_Flow_Graph g; g.setN(3); g.setST(1,3);
int id=g.add_edge(1,2,2,-1); g.add_edge(2,3,2,3);
auto [f,c]=g.min_cost_flow();
assert(f==2 && c==4 && g.edge_flow(id)==2);
```

使用边界

允许负费用边，但整个输入图不能有负费用环。容量、总流量及增广过程中的总费用须能用 $\mathbb{I}$ 表示；距离使用 `__int128`。初始 SPFA 最坏 $O(nm)$ ；设最短路费用阶段数为 R ，每阶段保守上界为 $O(n^2m + (n+m)\log(n+m+2))$ ，总耗时还需乘阶段数并加初始化开销，不是总计 $O(nm)$ 。空间 $O(n+m)$ 。

4.8 graph/flow/min-cost-flow-simplex.cpp

最小费用最大流

什么时候用

先求最大可行流量，再在最大流中最小化费用。使用网络单纯形：先建立残量生成森林，再通过环增广和生成树换基调整流量，复用未受换基影响的势能缓存。直接在原图残量边上计算，不设强制重启阈值，不使用递归。人工边费用取原图边权绝对值之和加一；证明势能运算不会溢出时使用 `ll`，否则使用 `__int128`。

调用顺序与接口

- `Cost_Flow_Graph g; g.setN(n); g.setST(s,t)` 初始化。
- `add_edge(u,v,cap,cost)` 添加有向边并返回正向边编号。
- `min_cost_flow()` 返回 {总流量, 总费用}; `edge_flow(id)` 查询流量。

最小示例（放入 main）：

```
Cost_Flow_Graph g; g.setN(3); g.setST(1,3);
int id=g.add_edge(1,2,2,-1); g.add_edge(2,3,2,3);
auto [f,c]=g.min_cost_flow();
assert(f==2 && c==4 && g.edge_flow(id)==2);
```

使用边界

允许负费用边，但输入图不能有负费用环。没有指定流量参数；要限制流量，增加一个容量为上限的新源边。更改图后从原始容量重算，不改图返回缓存。容量、总流量和费用答案须能用 `ll` 表示；只汇总原图边费用，避免人工边的大费用参与乘法累加。每条边流量均可查询。运行时间取决于换基次数，退化数据仍可能较慢；空间为 $O(n + m)$ 。

4.9 graph/matching/bipartite.cpp

二分图最大匹配

什么时候用

给定左右两部，多次加边后求最大基数匹配，返回匹配的点对。先贪心匹配，再用多源 BFS 记录起点与前驱，遇到未匹配右点就立即增广；整个过程不递归。

调用顺序与接口

- `Bipartite_Matching g; g.setN(n,m)`，左部下标 $1..n$ ，右部下标 $1..m$ ；两部编号互不影响。
- `g.add_edge(u,v)` 加入左点 u 到右点 v 的边；允许重边。
- `auto a=g.solve()` 返回 `vector<pair<int,int>>`，每个二元组 (u,v) 是一对匹配的左点和右点，按左点递增；`a.size()` 就是最大匹配数。
- 求解后的 `matchL[u]`、`matchR[v]` 是对应另一部的点，未匹配为 0。`set()` 清空，`setN` 清空并设置两部大小。

最小示例（放入 main）：

```
Bipartite_Matching g; g.setN(3,3);
g.add_edge(1,1); g.add_edge(2,1);
g.add_edge(2,2); g.add_edge(3,2);
auto a=g.solve(); assert(a.size()==2);
g.add_edge(3,3); a=g.solve(); assert(a.size()==3);
for(auto [u,v]:a){
    assert(g.matchL[u]==v && g.matchR[v]==u);
}
```

使用边界

邻接表在 `solve` 内按左点压成连续存储，保持各点的加边顺序。`rt[u]` 记录搜索树的未匹配起点，`pre[u]` 记录增广前驱；某起点完成增广后，跳过该搜索树剩余队列项。一轮允许处理不同长度的增广路。设总点数 $V=n+m$ 、边数 E ，空间 $O(V+E)$ ，每轮 $O(n+E)$ ；保守最坏时间界为 $O((V+E)(\min(n,m)+1))$ ，不沿用分层 HK 的复杂度保证。空部、空边集允许，返回空匹配。每次 `solve` 都重新计算；可继续加边再调用。

4.10 graph/matching/general.cpp

无权一般图最大匹配

什么时候用

带花树，允许奇环，返回匹配边数，最坏 $O(n^3)$ 。

调用顺序与接口

- blossom g; g.setN(n) 后 add_edge 添加无向边。
- solve() 返回最大匹配的边数; match[u] 为匹配点，未匹配为 0。
- solve()*2==n 判定存在完全匹配。

最小示例（放入 main）:

```
blossom g; g.setN(4);
g.add_edge(1,2); g.add_edge(2,3);
g.add_edge(3,1); g.add_edge(3,4);
assert(g.solve()==2);
assert(g.match[g.match[1]]==1);
```

使用边界

这是无权匹配，不会最小化或最大化边权。点数为奇数时不可能完全匹配。重复 solve 会重新求解。

4.11 graph/matching/regular-bipartite.cpp

正则二分图的完美匹配分解

什么时候用

两部各 n 个点，每个点度数均为 d ，将全部 nd 条边恰好分成 d 组完美匹配。支持多重边，同一点对点之间的边按重数计入分解。

调用顺序与接口

- `Regular_Bipartite g; g.setN(n); g.add_edge(u,v)` 加入左点 u 到右点 v 的边，两部下标均为 $1..n$ 。
- `auto a=g.solve()` 返回 `vector<vector<int>>`，共 d 组。每组长度 $n+1$ ，`a[c][u]` 直接是第 c 组中左点 u 匹配的右点；`a[c][0]=0` 占位。
- 每组的 `a[c][1..n]` 都是右部点的一个排列。所有组中 (u,v) 出现的总次数等于该边的输入重数。组编号为 $0..d-1$ ，组间没有固定的字典序要求。
- d 从输入度数推得，`solve` 用 `assert` 检查正则条件； $d=0$ 返回空分解。 $n=0$ 也返回空分解。

最小示例（放入 main）：

```
Regular_Bipartite g; g.setN(2);
g.add_edge(1,1); g.add_edge(1,2);
g.add_edge(2,1); g.add_edge(2,2);
auto a=g.solve(); assert(a.size()==2);
for(auto &match:a){
    assert(match.size()==3 && match[0]==0);
    assert(1<=match[1] && match[1]<=2);
    assert(1<=match[2] && match[2]<=2);
    assert(match[1]!=match[2]);
}
```

使用边界

确定性算法，时间 $O(n + nd \log(nd + 1))$ ，包括结果在内空间 $O(n + nd)$ 。偶数度用欧拉分边；奇数度通过整数重数缩减找完美匹配，再借用已分解的匹配，把另一分支凑成二的幂次度数。无随机失败概率。非正则图不属于此接口，不要关闭断言后直接输入；重复 `solve` 不修改原边集。

4.12 graph/steiner-tree.cpp

最小斯坦纳树

什么时候用

无向非负权图中，求连接给定关键点的最小费用子图，允许使用其他点。设关键点去重后有 k 个，子集 DP 加多源 Dijkstra 的时间为 $O(n3^k + m \log(m + 2)2^k)$ ，空间为 $O(n2^k + n + m)$ 。固定一个关键点作为根，实际只为剩余 $k - 1$ 个点开子集状态。

调用顺序与接口

- `Steiner_Tree g; g.setN(n)` 初始化，下标 $1..n$ ；`set()` 清空。
- `add_edge(u,v,w)` 添加一条无向边，不要重复添加反向边。
- `solve(s)` 接收关键点的 `vector<int>`，返回最小费用；不连通返回 `-1`。

最小示例（放入 main）：

```
Steiner_Tree g; g.setN(4);
g.add_edge(1,4,2); g.add_edge(2,4,2); g.add_edge(3,4,2);
assert(g.solve({1,2,3})==6); // use vertex 4 as a Steiner point
assert(g.solve({1,1})==0);
```

使用边界

空关键点集或只有一个不同关键点时返回 0，重复关键点自动去重。支持零权、重边和自环，不支持负权。边权和有限答案必须小于 `LLONG_MAX`；中间状态超过范围时会被截断，不会有整数加法溢出。只返回费用，不恢复边集。每次 `solve` 独立计算，可更换关键点或加边后重算； k 较大时，先估算指数级时间和内存。

4.13 graph/two-sat.cpp

布尔约束求解

什么时候用

求满足所有二元或子句的一组赋值，时间与空间 $O(n+m)$ 。

调用顺序与接口

- `two_sat s; s.setN(n)` 初始化 n 个布尔变量，下标 $1..n$ 。
- `add(x,a,y,b)` 表示 (变量 x 等于 a) 或 (变量 y 等于 b)， a,b 为 0/1。
- `solve()` 无解返回空 `vector`，否则 `ans[1..n]` 为可行赋值。

最小示例（放入 main）：

```
two_sat s; s.setN(2);
s.add(1,1,1,1); // force x1=true
s.add(1,0,2,1); // x1 implies x2
auto ans=s.solve();
assert(!ans.empty() && ans[1] && ans[2]);
```

使用边界

只有“或”子句接口；例如强制 $x=a$ 可重复同一文字。`setN` 清空约束。不要把 `add` 当作直接添加两条蕴含边。

5 树 / tree

5.1 tree/cartesian-tree.cpp

小根堆笛卡尔树

什么时候用

$O(n)$ 构建，同时满足下标的中序顺序与值的小根堆序，空间 $O(n)$ 。

调用顺序与接口

`auto [ls,rs]=cartesian(a)` 接受从 0 开始的 `vector`，返回两个同长度 `vector`，分别为左右儿子下标。无儿子用 -1。相等值保留较早下标在上方。根是原数组中第一个最小值的位置。

最小示例（放入 `main`）：

```
vector<int> a{3,1,2,1};
auto [ls,rs]=cartesian(a);
int rt=min_element(a.begin(),a.end())-a.begin();
assert(rt==1 && ls[rt]==0 && rs[rt]==3 && ls[3]==2);
```

使用边界

空输入返回两个空 `vector`，此时没有根。构造无递归，单调数组形成长链也可直接构建；后续遍历要自行避免递归爆栈。类型只需支持小于比较。返回值不含原数组或额外哨兵。

5.2 tree/centroid-decomposition.cpp

点分治框架

什么时候用

提供寻找重心、递归处理剩余连通块的框架。与 HLD 查询共用 `Tree`，题目需要的统计尚未实现。

调用顺序与接口

- `Tree t; t.setN(n)` 后加入树边，设置 `rt`。
- 需要 LCA 时先 `HLD_build`；`DC_build` 从整棵树开始点分治。
- 在 `get` 中收集子树信息，在 `DC` 中实现合并、统计和清空逻辑。

粘贴本文件之前：

最小示例（放入 `main`）：

```
Tree t; t.setN(3);
t.add_edge(1,2); t.add_edge(2,3);
t.HLD_build(); assert(t.dist(1,3)==2);
t.DC_build(); // traversal only, no problem answer
```

使用边界

本例只演示初始化和遍历，不会输出路径统计答案。不能原样当作“距离为 k 的点对计数”使用。

5.3 tree/divide-combine-tree.cpp

析合树：返回完整结构

什么时候用

对 $1..n$ 的排列构建析合树。构建时间 $O(n \log n)$ ，包含四毛子 RMQ、辅助线段树和返回结构在内的空间均为 $O(n)$ 。

调用顺序与接口

先粘贴 data-structure/linear-rmq.cpp。xihe_tree tr; auto t=tr.build(a) 接受不带哨兵的 vector; 也可先 setN(n), 填写 tr.a[1..n], 再调用 build()。结果独立持有数据: rt 为根, cnt 为结点数, son[u] 为按位置排序的儿子; typ[u] 为 LEAF/INC/DEC/PRIME; [L[u],R[u]] 为下标闭区间, [mn[u],mx[u]] 为值域闭区间; id[i] 为第 i 项对应的叶子编号。

最小示例 (放入 main):

```
xihe_tree tr; auto t=tr.build(vector<int>{3,1,2});
int u=t.rt;
assert(t.L[u]==1 && t.R[u]==3);
assert(t.mn[u]==1 && t.mx[u]==3);
assert(t.typ[u]==xihe_tree::DEC && t.son[u].size()==2);
assert(t.mn[t.id[2]]==1);
```

使用边界

输入必须恰为 $1..n$ 的排列，不接受重复值；必要时在外部离散化，但需确认题目是否只关心排名连续。返回的结点编号、下标区间、id 均从 1 开始，0 不使用。n=0 时 rt=cnt=0。INC/DEC 是儿子值域顺序递增/递减的合点，PRIME 是析点。不要假定父编号大于儿子编号；在返回结果上自行遍历、DP，构建器不含 solve。再次 build 或修改构建器不会改变已返回结果。

5.4 tree/heavy-light-decomposition.cpp

LCA、祖先和路径分段

什么时候用

对一棵连通无向树预处理，支持 $O(1)$ LCA 和 $O(\log n)$ 路径分段。

调用顺序与接口

- `Tree t; t.setN(n)` 后加入 $n-1$ 条边；可设置 `t.rt`，再 `build`。
- `lca(u,v)`、`dist(u,v)` 返回最近公共祖先及边数距离；`kth_anc(u,k)` 的 $k=0$ 返回自身。
- `in[u]` 是 DFS 编号，`dfn[i]` 反查原点；子树为 `[in[u],out[u]]`。
- `index_of_path(u,v)` 返回覆盖点路径的一组 DFS 序闭区间。

最小示例（放入 main）：

```
Tree t; t.setN(3);
t.add_edge(1,2); t.add_edge(1,3); t.build();
assert(t.lca(2,3)==1 && t.dist(2,3)==2);
int cnt=0;
for(auto [l,r]:t.index_of_path(2,3))cnt+=r-l+1;
assert(cnt==3);
```

使用边界

配线段树时，把点 u 的值放在位置 `in[u]`，不要直接用 u 。路径段没有保证从 u 到 v 的方向顺序，只能直接用于可交换聚合。按边统计时要另行排除 LCA。与 `DC.cpp` 都定义 `Tree`。

6 字符串 / string

6.1 string/aho-corasick.cpp

多模式串 AC 自动机

什么时候用

一次加入多个模式串，再统计它们在文本中的出现次数。允许重叠出现；构建 $O(26 * \text{节点数})$ ，每次 count 为 $O(\text{文本长度} + \text{节点数})$ 。

调用顺序与接口

- AC ac; id=ac.insert(" "+pattern) 返回该模式串末节点编号。
- 插入完所有模式串后调用 build，之后不能再插入，除非 set 清空重建。
- auto cnt=ac.count(" "+text)，模式串答案是 cnt[id]。
- build 后 ch 是补全后的转移，可直接用于自动机 DP；fail 是失配链接。

最小示例（放入 main）：

```
AC ac; int a=ac.insert(" aba"),b=ac.insert(" ba");
ac.build(); auto cnt=ac.count(" ababa");
assert(cnt[a]==2 && cnt[b]==2);
```

使用边界

模式和文本都只支持小写字母，前导空格不参加匹配。重复模式串返回同一编号，但它们各自的答案仍是该模式在文本中的出现次数。空模式返回根编号 0，次数为文本长度 +1。不同 count 之间不会累计。

6.2 string/generalized-suffix-automaton.cpp

广义后缀自动机

什么时候用

维护多个小写串的所有子串；出现次数是这些串中次数之和，重复插入的串也重复计数，不允许跨串边界匹配。

调用顺序与接口

- `GSAM t; t.insert(" "+s)` 加入一条字符串，之后 `t.build()` 累计出现次数。
- `count(" "+p)` 返回模式的总出现次数；`find(" "+p)` 返回状态编号，不存在为 -1。
- `distinct()` 返回所有插入串合起来的不同非空子串数。
- 可继续 `insert`，但下一次 `count` 前必须重新 `build`。

最小示例（放入 main）：

```
GSAM t; t.insert(" aba"); t.insert(" bab");
t.build(); assert(t.count(" ab")==2);
assert(t.count(" abab")==0 && t.distinct()==6);
t.insert(" aba"); t.build(); assert(t.count(" aba")==2);
```

使用边界

`count` 不是“包含该子串的字符串个数”。根为 0，`fail[0]=-1`；别把普通 SAM 简单重置 `lst` 的做法当成这里的插入实现。`set(m)` 的 `m` 用于预留总字符量级的空间，`build` 的计数按当前全部插入内容重新计算。

6.3 string/kmp-z.cpp

前缀函数和 Z 函数

什么时候用

线性时间计算字符串的 `border` 与各后缀和全串的公共前缀长度。

调用顺序与接口

- `border(" "+s)` 返回 `pi`，`pi[i]` 为 `s[1..i]` 的最长真 `border` 长度。
- `z_function(" "+s)` 返回 `z`，`z[i]` 为 `s[i..n]` 与 `s[1..n]` 的 LCP。
- 数组长度 `n+1`，0 号是占位。

最小示例（放入 main）：

```
auto pi=border(" ababa"); auto z=z_function(" ababa");
assert(pi[5]==3 && z[3]==3);
```

使用边界

这个实现 `z[1]=0`，而不是全串长度。没有封装“模式串在文本的所有位置”：可自行 KMP 扫描，或拼接 `pattern+ 分隔符 +text` 后用 `Z`；分隔符不能出现在两个原串里。

6.4 string/lyndon.cpp

Lyndon 分解

什么时候用

用 Duval 将串唯一分解为字典序不增的 Lyndon 串。Lyndon 串严格小于自己的每个非空真后缀，相同因子可以连续出现。

调用顺序与接口

`auto a=lyndon(s)`; `s` 带前导空格。返回 `vector<pair<int,int>>`，各项为一个因子的 1 下标闭区间 `[l,r]`，按原串从左到右排列且恰好覆盖全串。需要内容时用 `s.substr(l,r-l+1)`。

最小示例（放入 main）：

```
string s=" ababab"; auto a=lyndon(s);
assert((a==vector<pair<int,int>>{{1,2},{3,4},{5,6}}));
assert((lyndon(" cba")==vector<pair<int,int>>{{1,1},{2,2},{3,3}}));
assert(lyndon(" ").empty());
```

使用边界

时间 $O(n)$ ，除返回区间外额外空间 $O(1)$ 。比较按 `unsigned char` 的字节顺序，可处理任意字节；不要小写字母。同样的因子不会合并，`ababab` 分为三个 `ab`，而 `aaaa` 分为四个 `a`。空串返回空向量。本接口是整串分解，不是每个位置的最长 Lyndon 前缀数组。

6.5 string/manacher.cpp

Manacher 回文半径

什么时候用

线性求出所有奇数、偶数长度回文的最大半径，并以 $O(1)$ 判断某个子串是否回文。

调用顺序与接口

- `Manacher t; t.build(s);` `s` 带前导空格，实际下标 `1..n`；`set()` 清空。
- `d1[i]` 包含中心字符：最长奇回文为 `[i-d1[i]+1,i+d1[i]-1]`，长度 `2*d1[i]-1`。
- `d2[i]` 的中心在 `i-1` 与 `i` 之间：最长偶回文为 `[i-d2[i],i+d2[i]-1]`，长度 `2*d2[i]`，半径可以为 0。
- `ask(l,r)` 判断非空闭区间是否回文，要求 $1 \leq l \leq r \leq n$ 。所有回文出现次数为各位置 `d1+d2` 的总和，不是不同回文数。

最小示例（放入 main）：

```
Manacher t; t.build(" abba");
assert(t.d2[3]==2 && t.d1[2]==1);
assert(t.ask(1,4) && t.ask(2,3));
assert(!t.ask(1,3));
long long cnt=0;
for(int i=1;i<=t.n;i++)cnt+=t.d1[i]+t.d2[i];
assert(cnt==6);
```

使用边界

时间、空间均为 $O(n)$ ；不插入分隔字符，可处理任意字节。奇偶半径含义不同，不能直接把 `d1` 当成回文长度。空串允许 build，但没有可查询的非空区间。求最长回文取所有 `2*d1[i]-1` 与 `2*d2[i]` 的最大值，空串答案为 0。

6.6 string/palindromic-automaton.cpp

回文自动机

什么时候用

在线维护不同回文子串、最长回文后缀及回文出现次数。字符集为小写字母，建树与空间均为 $O(n)$ 。

调用顺序与接口

- PAM `p`; `p.build(s)` 从整串重建; `s` 带前导空格。也可 `p.set(n)` 预留容量, 再逐字 `p.extend(c)`, 返回当前最长回文后缀结点。
- 0 是长度 0 的根, 1 是长度 -1 的根; 真正的回文结点为 `2..tot`, 不同回文数为 `tot-1`。 `lst` 是当前最长回文后缀。
- `len[u]` 为长度, `fail[u]` 为最长真回文后缀; `num[u]` 为该回文的非空回文后缀个数。因此每次加入后的 `num[lst]` 是当前前缀末尾的回文数。
- `pos[u]` 为首次出现的右端点, 首次区间是 `[pos[u]-len[u]+1, pos[u]]`。 `cnt=p.count()` 返回每个回文的总出现次数, 可重叠; 根的次数置 0。

最小示例 (放入 main):

```
PAM p; p.build(" ababa");
auto cnt=p.count();
assert(p.tot-1==5 && p.len[p.lst]==5);
assert(p.num[p.lst]==3 && cnt[p.lst]==1);
int u=p.ch[1]['a'-'a']; assert(cnt[u]==3);
p.extend('b'); cnt=p.count(); // still safe after count()
assert(p.len[p.lst]==5 && cnt[u]==3);
```

使用边界

`siz` 只记录结点作为最长后缀的次数, 不能直接作为总出现次数。 `count` 在副本上沿 `fail` 累加, 单次 $O(n)$, 可以重复调用, 也能继续 `extend`; 不要每加入一个字符都调用 `count`。所有回文出现次数之和用 `ll` 保存。空串写作一个空格。

6.7 string/rolling-hash.cpp

双模前缀哈希

什么时候用

提供子串指纹与拼接，不是无碰撞的字符串判等。构建前缀数组 $O(n)$ ，常规子串取值 $O(1)$ 。

调用顺序与接口

- `auto h=Hash_of(" "+s)` 产生前缀哈希。
- 子串 $[l,r]$ 为 $h[r]-h[l-1]$ 。
- `a+b` 拼接两段，`h.v()` 打包哈希值；`.l` 是长度。
- `init_hash(n)` 可预留幂次；实际操作也会自动扩容。

最小示例（放入 main）：

```
auto h=Hash_of(" ababa");
auto a=h[3]-h[0],b=h[5]-h[2];
assert(a.l==b.l && a.v()==b.v());
```

使用边界

比较子串时同时比较长度和哈希。底数每个进程随机，不能把数值用于跨进程永久存储。减法只表示删去前缀，不是任意删除；Hash 对象应使用 `{}` 零初始化。

6.8 string/runs.cpp

Runs 极大周期子串

什么时候用

枚举所有长度至少为最小周期两倍、且不能保持该周期向左或向右扩展的子串。返回若干三元组 (l,r,p) ，其中 p 必须是最小周期。

调用顺序与接口

先抄 `data-structure/linear-rmq.cpp`，再抄 `string/suffix-array.cpp`，再抄本文件。调用 `Runs t; auto a=t.build(s)`，返回 `vector<array<int,3>>`，依次为 l 、 r 、 p 。输入有前导空格，区间为 1 下标闭区间，结果按三元组字典序排序并去重。空串、单字符及无重复的串返回空向量。

最小示例（放入 main）：

```
Runs t; auto a=t.build(" ababab");
assert((a==vector<array<int,3>>{{1,6,2}}));
a=t.build(" aabaa");
assert((a==vector<array<int,3>>{{1,2,1},{4,5,1}}));
assert(t.build(" abc").empty());
```

使用边界

实际字符限字节 1..255,0 留给 SA 哨兵。采用两种字母序下的最长 Lyndon 前缀，加 `SA_LCP`、`SA_LCS` 向两侧扩展；LCP 使用四毛子，不使用哈希。源码版时间 $O(n \log n)$ （最后排序），空间 $O(n)$ ；考场倍增 SA 也满足此界。Runs 不包括所有周期区间，只给出极大区间；长度不必是 p 的倍数，例如 `ababa` 返回 $(1,5,2)$ 。

6.9 string/suffix-array.cpp

后缀数组、LCP 与 LCS

什么时候用

先粘贴 data-structure/linear-rmq.cpp。SA-IS 构建后缀排序，SA_LCP 用 height 和线性 RMQ 查询后缀公共前缀；SA_LCS 在反串上查询两个前缀的公共后缀。

调用顺序与接口

- SA sa; sa.build(" "+s) 输入前面补一个占位空格，真实字符下标 1..n。
- sa.sa[r] 是排名第 r 的后缀起点；sa.rk[i] 是后缀 i 的排名。
- SA_LCP l; l.build(sa)，然后 l.lcp(i,j) 或 l.ask(i,j) 查询，i,j 是原串位置。
- 也可 l.build(" "+s) 直接建。height[r] 为排名 r 与 r-1 的后缀 LCP，height[1]=0。
- SA_LCS r; r.build(sa) 或 r.build(" "+s); r.lcs(i,j) / r.ask(i,j) 查询以 i、j 结尾的最长公共后缀，允许 i 或 j 为 0，此时返回 0。

最小示例（放入 main）：

```
SA sa; sa.build(" banana");
assert(sa.sa[1]==6);
SA_LCP l; l.build(sa);
assert(l.lcp(2,4)==3); // anana / ana
assert(l.lcp(3,3)==4);
SA_LCS r; r.build(sa);
assert(r.lcs(4,6)==3); // bana / banana
assert(r.lcs(0,6)==0);
```

使用边界

SA、LCP、LCS 均 $O(n)$ 构建和空间，查询 $O(1)$ 。LCS 参数是原串前缀的右端点。不要把后缀排名传给 lcp。默认字符编码范围 1..127，任意非零字节传 M=256；不支持串内零字节。修改原串后须重建相关结构。

6.10 string/suffix-automaton.cpp

单串后缀自动机

什么时候用

构建单个小写字母串的子串状态图，并统计每个状态对应子串的出现次数。时间及空间 $O(n)$ ，字母表固定 26。

调用顺序与接口

- SAM t; t.build(" "+s) 每次重新构建。
- 根为 0; ch[u][c] 为字符 c 的转移，0 表示不存在。
- 状态 u 对应长度区间 (len[fail[u]],len[u])，同一状态内的子串出现次数均为 siz[u]。

最小示例（放入 main）：

```
SAM t; t.build(" ababa");
int u=0; for(char c:string("aba"))u=t.ch[u][c-'a'];
assert(u && t.siz[u]==2);
ll distinct=0;
for(int i=1;i<=t.tot;i++)distinct+=t.len[i]-t.len[t.fail[i]];
assert(distinct==9);
```

使用边界

匹配模式串时遇到不存在转移应立即判定不出现。build 内已累计 siz，不能再次累计。裸调用 extend 不会自动维护整套出现次数；多串应用请用 GSAM。solve 是“重复子串长度乘次数最大值”的特定示例，不是通用查询。

7 数学 / math

7.1 math/big-int.cpp

高精度有符号整数

什么时候用

十进制大整数，vector 按 10^9 分块。加减 $O(n)$ ，普通乘法及长除法最坏 $O(nm)$ ， n 、 m 是分块数。

调用顺序与接口

- `BigInt x=123` 从 `long long` 构造；更长整数用流读入。
- 支持 `+` `-` `*` `/` `%`、复合赋值和比较。
- `a.divmod(b)` 同时返回商和余数，避免分别执行除法与取模。

最小示例（放入 main）：

```
BigInt a,b;
istringstream in("-12345678901234567890 100");
in >> a >> b;
auto [q,r]=a.divmod(b);
assert(q*b+r==a);
ostringstream out; out << q << ' ' << r;
assert(out.str()=="-123456789012345678 -90");
```

使用边界

除法向零取整，余数与被除数同号；除数不能为零。这不是 FFT 高精度乘法，也不支持小数。

7.2 math/dynamic-mod-int.cpp

运行时模数及组合数

什么时候用

同一份程序需要选定不同模数时使用。设置模数后再创建该轮使用的值。

调用顺序与接口

- `mint::setM(p)` 修改模数。
- `init_fact(n)` 重建 $0..n$ 阶乘、逆阶乘，要求 $n!$ 可逆。
- `C(k,n)` / `binom(n,k)` 查询前必须已预处理到 n 。

最小示例（放入 main）：

```
mint::setM(101); init_fact(10);
assert(C(2,5).x==10);
mint a=100; assert((a+2).x==1);
```

使用边界

修改模数不会转换已有 `mint` 变量；丢弃旧值并重建阶乘。与固定模数版、各多项式版本的 `mint` / `fac` 等全局名字冲突。

7.3 math/gaussian-elimination.cpp

线性方程组与行列式

什么时候用

用消元求 $Ax=b$ 的一组解，或计算方阵行列式。支持质数模数下的 `mint` 以及浮点类型；不能用 `int/ll` 实例化。

调用顺序与接口

- `Gauss<T> g; res=g.solve(a,m)`，`a` 是 n 行 $m+1$ 列的增广矩阵，最后一列为 `b`。
- 返回 -1 无解 (`x` 被清空)，0 多解，1 唯一解；解存 `g.x`，长度 `m`，下标 `0..m-1`。
- 多解时自由变量取 0；`g.rank` 是系数矩阵秩，`where[j]=-1` 表示自由变量。
- `g.det(a)` 接受 $n*n$ 方阵，返回 `T`，不改变 `x/rank`。空方阵行列式为 1。

最小示例（放入 `main`）：

```
Gauss<long double> g;
// x+y=3, 2x-y=0
int res=g.solve({{1,1,3},{2,-1,0}},2);
assert(res==1 && abs(g.x[0]-1)<1e-9L);
assert(abs(g.x[1]-2)<1e-9L);
assert(abs(g.det({{1,2},{3,4}})+2)<1e-9L);
```

使用边界

`Gauss<mint>` 需要先粘贴 `ModInt` 或一个内置 `mint` 的多项式文件。浮点使用最大绝对值选主元和绝对阈值 `eps`（默认 `1e-12`），尺度差异大或病态方程需要自行缩放并检查残差。方阵时间 $O(n^3)$ ，复制矩阵后原输入保持不变；多个 RHS 要自行扩展。

7.4 math/linear-basis/field-prefix.cpp

前缀模意义线性基 调用顺序与接口

先抄 `ModInt`。 `Prefix_Linear_Basis<mint> B; B.setN(n)`，依次 `insert(x,id)`。 `id` 必须为严格递增的正整数；增秩返回 0，否则返回最早可替换元素的编号。规则与前缀异或版相同，只是异或消元换成域上的线性消元。

最小示例（放入 `main`）：

```
Prefix_Linear_Basis<mint> B; B.setN(2);
assert(B.insert({1,0},1)==0);
assert(B.insert({0,1},2)==0);
assert(B.insert({0,2},3)==2);
assert(B.insert({1,2},4)==1);
assert(B.get_rank(3)==2 && B.contains({1,0},3));
assert(!B.contains({1,0},4));
assert(B.insert({0,0},5)==5);
```

使用边界

处理到编号 `r` 时，`get_rank(l)` 查询 `[l,r]` 的秩，`contains(x,l)` 判断 `x` 是否在该区间张成空间中。`l` 默认 1，超出 `r` 时为空基。插入和可表示性查询 $O(n^2)$ ，秩查询 $O(n)$ ，空间 $O(n^2)$ 。只用于精确域，通常用质数模数 `mint`；`setN` 清空所有状态。`pos` 保留原编号，`a` 是消元后的向量。不是任意删除或持久化结构。

7.5 math/linear-basis/field.cpp

模意义线性基 调用顺序与接口

先抄一份 `ModInt`； `Linear_Basis<mint> B; B.setN(n)` 设置向量维数。`insert(x)` 返回是否增秩；`contains(x)` 判断 `x` 是否在张成空间中；`rank` 是当前秩。输入 `vector` 长度必须为 `n`，坐标下标 `0..n-1`。

最小示例（放入 `main`）：

```
Linear_Basis<mint> B; B.setN(2);
assert(B.insert({1,0}) && B.insert({0,1}));
assert(!B.insert({2,3}) && B.rank==2);
assert(B.contains({7,9}));
B.setN(2); assert(!B.contains({1,0}));
```

使用边界

`T` 要求精确域上的运算，通常使用质数模数 `mint`；不支持普通 `int/ll`、合数模数或浮点数。插入和查询 $O(n^2)$ ，空间 $O(n^2)$ ，占用的主元行按实际长度存储。`n=0` 允许，空向量不增秩。只维护张成空间，不返回具体线性组合系数。

7.6 math/linear-basis/xor-prefix.cpp

前缀异或线性基

调用顺序与接口

Prefix_Xor_Basis B; B.setM(m) 设置位数并清空，默认 64 位，允许 0 至 64 位。按加入顺序调用 insert(x,id)，id 必须为严格递增的正整数，可不连续。增秩返回 0，否则返回本次相关关系中最早且可删除的元素编号；零向量返回自身编号。set() 保留位数，清空并重置编号；setM(m) 同时重设位数和编号。

最小示例（放入 main）：

```
Prefix_Xor_Basis B; B.setM(2); // bits 0..1
assert(B.insert(1,1)==0 && B.insert(2,2)==0);
assert(B.insert(2,3)==2); // removing id=1 would lower rank
assert(B.insert(3,4)==1);
assert(B.get_rank(3)==2 && B.contains(1,3));
assert(!B.contains(1,4) && B.ask(0,4)==3);
assert(B.insert(0,5)==5);
```

使用边界

处理到右端点 r 后，get_rank(l)、contains(x,l)、ask(x,l) 分别查询编号 [l,r] 的秩、可表示性和最大异或值。l 默认 1；l 大于 r 时视为空基。rank 是全部已插入元素的秩。pos 中的正编号是保留的原元素编号，a 是消元后的基。插入值必须满足 $x < 2^m$ (m=64 时允许任意 ull)。单次 O(m)，空间 O(m)；查询历史右端点须离线按 r 扫描或自己保存副本，没有持久化。contains(x,l) 对超范围高位返回 false，ask(x,l) 保留 x 的高位。

7.7 math/linear-basis/xor.cpp

异或线性基

调用顺序与接口

维护 m 位无符号整数的异或张成空间，默认 m=64；setM(m) 设置位数并清空， $0 \leq m \leq 64$ ，使用位 0..m-1。Xor_Basis B 默认空基；set() 清空并保留位数，insert(x) 返回是否增秩，rank 为当前秩。contains(x) 判断能否异或得到 x；ask(x=0) 返回 x 与基中任意子集异或的最大值，允许空集。

最小示例（放入 main）：

```
Xor_Basis B; B.setM(3); // bits 0..2
assert(B.insert(1) && B.insert(2));
assert(!B.insert(3) && B.rank==2);
assert(B.contains(3) && !B.contains(4));
assert(B.ask()==3 && B.ask(4)==7);
```

使用边界

插入、查询均 O(m)，空间 O(m)。插入值必须满足 $x < 2^m$ ，m=64 时允许任意 ull。contains(x) 对带有超范围高位 x 返回 false；ask(x) 允许这样的 x，高位保持不变。输入按无符号大小比较；m=64 时第 63 位可用，m=0 时只能插入零。零向量不增秩。a 中存的是消元后的向量，不是原输入元素。

7.8 math/mod-int.cpp

固定模数及组合数

什么时候用

默认模数 998244353。模数写在源码中的 MOD；阶乘表按需扩容。

调用顺序与接口

- mint 支持四则、pow(k)、inv()；除数必须可逆。
- init(n) 保证 fac、ifac、inv 覆盖到 n，组合数调用会自动预处理。
- C(k,n) 和 binom(n,k) 都表示从 n 个中选 k 个；这里 C 的参数顺序与常见记法相反。

最小示例（放入 main）：

```
init(10);
assert(C(2,5).x==10 && binom(5,2).x==10);
mint a=3; assert((a*a.inv()).x==1);
assert(a.pow(4).x==81);
```

使用边界

阶乘方案用于质数模数且 $n < \text{MOD}$ ；比较时可读 .x，此版未定义 mint 的 ==。输入运算符直接读 .x，读负数或未归一化数据时先读 long long 再构造 mint。避免 pow(LLONG_MIN)。多项式文件已内置本类，不要重复粘贴。

7.9 math/rational.cpp

有理数

什么时候用

用 ll 分子 p、分母 q 表示精确有理数，适合需要精确比较和运算的斜率、比例等。构造和运算后自动约分，分母保持正数，零统一为 0/1。比较 O(1)，运算的约分需要一次辗转相除。

调用顺序与接口

Rat a(p,q)，默认值为 0，Rat(p) 表示整数；支持四则运算、复合赋值、取负、六种比较及 ostream 输出。可以和整数直接运算，输出始终为 p/q。读取成员 a.p、a.q 可取得最简形式，不要直接修改。

最小示例（放入 main）：

```
Rat a(1,2),b(2,3);
assert(a+b==Rat(7,6) && a*b==Rat(1,3));
assert(a<b && 2*a==1);
a+=b;
ostringstream out; out<<a;
assert(out.str()=="7/6");
assert(Rat(2,-4)==Rat(-1,2));
```

使用边界

依赖公共底稿中的 ll。分母及除数不能为零；初始分子、分母和约分后的分子、正分母必须能存入 ll。中间交叉乘法及加减使用 __int128，因此允许中间值超出 ll 后约分回来，但最终溢出会触发 assert。没有浮点转换，也不表示无穷大。

7.10 math/xor-equations.cpp

XOR 方程组

什么时候用

解若干条形如 $x_0 \oplus x_2 = k$ 的方程。每个未知数是一个 `ll`，系数只决定该未知数是否参加异或；对系数 `bitset` 做消元时，同时异或整个右端 `ll`，一次处理全部 64 位。支持负数右端，按其位模式参与异或。

调用顺序与接口

- `Xor_Gauss<N> g`: `N` 是 `bitset` 的编译期容量，默认有 `N` 个变量。
- `setN(n)` 设置实际变量数，要求 $0 \leq n \leq N$ ，同时清空方程；`set()` 清空方程但保留 `n`。
- `add(a,b)`: `a` 为 `bitset<N>`，`b` 为 `ll`。`a[i]=1` 表示该行包含变量 `x[i]`，下标 `0..n-1`；更高的位必须为 0。返回整个方程组是否仍有解。
- `solve()` 返回 -1 无解、0 多解、1 唯一解。答案放在 `x[0..n-1]`，自由变量取 0；无解时 `x` 为空。
- 有解时 `rank` 为系数矩阵的秩，自由变量数为 `n-rank`。添加方程会清空旧答案，需要重新 `solve`。

最小示例（放入 `main`）:

```
Xor_Gauss<3> g;
g.add(bitset<3>(0b011),5); // x[0] ^ x[1] = 5
g.add(bitset<3>(0b110),3); // x[1] ^ x[2] = 3
assert(g.solve()==0);      // free x[2] = 0
g.add(bitset<3>(0b001),7); // x[0] = 7
assert(g.solve()==1);
assert(g.x==vector<ll>({7,2,1}));
assert(!g.add(bitset<3>(),1)); // 0 = 1
assert(g.solve()==-1);
```

使用边界

`bitset` 的最低位对应 `x[0]`；同一个变量在一条方程中出现两次会抵消，构造时可用 `flip(i)`。零系数行右端非零即矛盾。本模板使用 GNU `bitset` 的置位遍历接口。设 $W = \lceil N/64 \rceil$ ，每次插入最坏 $O((n+1)(W+1))$ ，回代 $O(n(n+W+1))$ ，存储上界 $O(N(W+1))$ 。`N` 与 `n` 同阶时可按常规 `bitset` 消元估算开销。

8 数论 / number-theory

8.1 number-theory/dujiao-sieve.cpp

μ 、 φ 前缀和

什么时候用

预筛小范围并缓存整除分块递归，分别查询 $\sum \mu(i)$ 、 $\sum \varphi(i)$ ， $i=1..n$ 。

调用顺序与接口

- `du_jiao d; d.init_mu(B)` 后调用 `sum_mu(n)`，返回 `ll`。
- `init_phi(B)` 后调用 `sum_phi(n)`，返回 `__int128`。
- $B \geq 1$ ， $n \geq 0$ ；两个初始化和缓存相互独立，可只使用一个。

最小示例（放入 `main`）：

```
du_jiao d; d.init_mu(3);
assert(d.sum_mu(10)==-1);
d.init_phi(3); assert(d.sum_phi(10)==32);
assert(d.sum_mu(0)==0 && d.sum_phi(0)==0);
```

使用边界

必须先初始化对应函数。 B 是预处理上界而非查询上限，常从 $n^{2/3}$ 附近结合内存选择；预筛 $O(B)$ ，递归性能随 B 和查询集改变。重复初始化会清空该项缓存。

8.2 number-theory/floor-sum.cpp

类欧几里得整除求和

什么时候用

计算闭区间 $i=l..r$ 上 $\text{floor}((a*i+b)/c)$ 的和，时间 $O(\log c)$ ，返回 `__int128`。

调用顺序与接口

- `floor_sum(l,r,a,b,c)` 参数都是 `ll`。
- l,r,a,b 均非负， $c>0$ ； $l>r$ 返回 0。

最小示例（放入 `main`）：

```
auto ans=floor_sum(1,4,3,1,2);
assert(ans==16); // 2+3+5+6
```

使用边界

不能直接处理负系数或负下标。输出可配 `int128_IO`；不要先在 `ll` 中计算 $a*l$ 。答案和内部累计值须在有符号 128 位范围内。

8.3 number-theory/linear-sieve.cpp

线性筛素数

什么时候用

$O(n)$ 时间和空间，适合需要枚举范围内全部素数。

调用顺序与接口

- `init(n)` 重建范围 $0..n$, $n \geq 1$ 。
- `is_prime[x]` 判断 x 是否为素数；`prime[1..m]` 为升序素数，`prime[0]` 是占位。

最小示例（放入 `main`）：

```
init(20);
assert(m==8 && prime[1]==2 && prime[m]==19);
assert(is_prime[17] && !is_prime[1]);
```

使用边界

不是任意大整数判素接口，不能访问 n 以外。全局 `init`、`m`、`is_prime` 名字较通用，与其他模板组合时应检查冲突。

8.4 number-theory/modular-arithmetic.cpp

模逆元和扩展 CRT

什么时候用

提供 `gcd`、`lcm`、逆元与不要求模数互素的同余方程合并。

调用顺序与接口

- `inv(a,m)` 要求 $m > 0$ 且 $\gcd(a,m)=1$ 。
- `COE(r,m)` 表示 $x=r \pmod m$ ， $a+b$ 或 $a+=b$ 合并两个方程。
- 结果 `empty()` 为 `true` 表示无解；否则 r 是标准余数、 M 是合并模数，`accept(x)` 判断满足性。

最小示例（放入 `main`）：

```
COE a(2,3),b(3,5); auto c=a+b;
assert(!c.empty() && c.r==8 && c.M==15);
assert(c.accept(23) && inv(3,11)==4);
assert((COE(0,2)+COE(1,2)).empty());
```

使用边界

合并后的模数须能放入 `ll`，内部乘法虽提升到 `__int128`，但结果不是任意精度。`inv` 函数会与部分模数模板的全局 `inv` 数组重名，需要按实际组合改名。

8.5 number-theory/pollard-rho.cpp

大整数判素与质因数分解

什么时候用

Miller-Rabin 配合 Pollard-Rho, 适用于范围内不适合筛到上界的整数。随机算法的运行时间依赖输入。

调用顺序与接口

- `is_prime(n)` 判断素数。
- `factor(n)` 返回升序质因子, 含重数; `n=1` 返回空。
- `pollard_rho(n)` 提供一个因子, 素数时返回自身。

最小示例 (放入 `main`):

```
auto f=factor(360);
assert((f==vector<ull>{2,2,2,3,3,5}));
assert(is_prime(1000000007ULL));
```

使用边界

`factor` 的输入要求 $1 \leq n < 2^{62}$, 不能传 0。不是完整 `uint64` 范围版本。判素函数与线性筛的 `is_prime` 数组同名, 不能原样同贴。

8.6 number-theory/stern-brocot.cpp

有理数二分

什么时候用

在 $0 \leq p \leq M$, $1 \leq q \leq M$ 的最简分数中寻找目标或目标两侧相邻分数。先粘贴 math/rational.cpp。沿 Stern–Brocot 树倍增跳步，再二分跳步长度；询问次数及本地计算量均为 $O(\log(M+1))$ ，额外空间 $O(1)$ ，不计判定函数内部开销。

调用顺序与接口

- SBT t; t.setM(M) 清空并开始搜索， $1 \leq M \leq \text{LLONG_MAX}$ 。
- 在 !t.done() 时调用 Rat x=t.ask() 获取分数，然后 t.tell(res): -1 表示 x 偏小，0 表示恰好，1 表示 x 偏大。反复 ask 不推进搜索。
- done() 表示搜索结束。若 found() 为真，l==r 是答案；否则 l 是最大的偏小候选，r 是最小的偏大候选。
- l、r 为 pair<ll,ll>，分别保存分子、分母。{-1,0} 表示没有偏小候选，{1,0} 表示没有偏大候选；这两个哨兵不能构造 Rat。

最小示例（放入 main）：

```
SBT t; t.setM(100); Rat target(2,3);
while(!t.done()){
    Rat x=t.ask();
    t.tell(x<target?-1:x>target?1:0);
}
assert(t.found() && t.l==make_pair(211,311));
// Bracket sqrt(2), which is not a rational candidate.
t.setM(10);
while(!t.done()){
    Rat x=t.ask();
    t.tell((__int128)x.p*x.p<2*(__int128)x.q*x.q?-1:1);
}
assert(!t.found());
assert(t.l==make_pair(711,511) && t.r==make_pair(1011,711));
```

为什么只有一个 log：一段倍增加二分确定跳步长度 k ，花费 $O(\log(k+1))$ 次询问。设两端的“分子加分母”为 $u、v$ ；完整一段后变为 $u+kv、u+(k+1)v$ （或交换），乘积至少放大 $k+1$ 倍。该乘积从 1 开始，始终不超过 $4M^2$ ，所以各段 $\log(k+1)$ 之和为 $O(\log M)$ ；最后提前终止的一段也只需 $O(\log M)$ 。查询分数天然最简，无需每次求 gcd。

使用边界

反馈必须对应同一个目标或单调分界，符号描述的是当前询问值，不能反过来。目标可以不在候选集合，也可以小于零或大于 M 。若只提供 -1 和 1，则搜索最终返回分界两侧候选，不会报告 found；对“< 与 >=”判定，l 是最大的 < 候选，r 是最小的 >= 候选，r 可以恰好等于目标。没有有限上界时 r 为 1/0；目标小于零时 l 为 -1/0，r 为 0/1。

9 多项式 / polynomial

9.1 polynomial/fwt.cpp

按位 AND / OR / XOR 卷积

什么时候用

求 $c_k = \sum_{i \oplus j = k} a_i b_j$, 其中 $\oplus$ 为按位 AND、OR 或 XOR。长度 $n = 2^k$, 每次变换 $O(n \log n)$, 额外空间 $O(1)$ 。

调用顺序与接口

- `convolution_and(a,b)`、`convolution_or(a,b)`、`convolution_xor(a,b)` 直接返回对应卷积, 不修改传入数组。支持长度不同、非二次幂长度; 任一输入为空则返回空数组, 否则结果长度为不小于两者最大长度的最小二次幂。
- `fwt_and(a)`、`fwt_or(a)`、`fwt_xor(a)` 原地正变换; 第二个参数传 `true` 做逆变换。
- 支持 `vector<int>`、`vector<ll>` 和 `vector<mint>`。mint 需先抄模整数定义, FWT 本身不依赖多项式文件。
- 两个数组补零到相同的非零二次幂长度 n , 使 n 大于最大下标; 不必补到两者长度之和。
- 两边做同一种正变换, 逐点相乘, 再做对应逆变换, 即得卷积。

最小示例 (放入 main):

```
vector<int> a{1,2},b{3,4};
auto c=convolution_and(a,b); assert((c==vector<int>{13,8}));
c=convolution_or(a,b); assert((c==vector<int>{3,18}));
c=convolution_xor(a,b); assert((c==vector<int>{11,10}));
// vector<mint> works with the same interfaces.
```

使用边界

下标从 0 开始。int/ll 表示普通整数运算, 不会自动取模, 正变换和逐点乘法的中间值也必须不溢出。整数 XOR 逆变换逐层除以 2, 要求每步可整除, 不能把任意整数数组当成合法频域数据。XOR 模运算要求 2 可逆, 通常使用奇数模数; AND/OR 无此限制。不需要归一化第二次。

9.2 polynomial/integer.cpp

整数卷积与递推第 i 项

什么时候用

需要精确 long long 答案时使用。内部在三个质数下计算并通过 CRT 恢复有符号整数，不设置目标模数，也不使用浮点 FFT。递推仅要求返回项能存入 ll，中间生成函数的整数系数可以超过 ll。

调用顺序与接口

- `poly / Poly` 为 `vector<ll>`。 `f*g` 返回普通整数卷积，支持负数；任一输入为空则返回空向量。
- `recurrence(a,c,i)`: 已知 d 个初值和 d 个系数， $a_n = \sum_{j=0}^{d-1} c_j a_{n-1-j}$ ，返回第 i 项。两个向量等长， i 从 0 开始；空向量表示零数列。
- `FSPE(F,G,i)` 返回生成函数 F/G 的第 i 项，要求 G 非空且 $G[0]$ 为 1 或 -1， F 可以为空。
- `RSPE(a,i)` 从样本推断递推再求第 i 项。必须已知数列满足 d 阶整数系数齐次线性递推，提供至少 $2d$ 个连续初始项；空样本约定为零数列。 i 在样本范围内直接返回该项。

最小示例（放入 main）:

```
poly f{1,-2,3},g{4,5};
assert((f*g==poly{4,-3,2,15}));
assert(recurrence({0,1},{1,1},10)==55);
assert(FSPE({0,1},{1,-1,-1},10)==55);
assert(RSPE({0,1,1,2,3,5,8,13},92)==7540113804746346429LL);
// Large intermediate coefficients, constant answer.
assert(recurrence({1,1},{LLONG_MAX,1-LLONG_MAX},
                  1000000000000000000LL)==1);
// A genuinely signed 64-bit result.
assert((poly{LLONG_MIN}*poly{1}==poly{LLONG_MIN}));
```

使用边界

真实卷积的每个系数、递推的返回项必须在 ll 范围内。CRT 可在这个范围内唯一恢复答案；断言能发现部分越界，但不能作为任意大结果的溢出检测器。单次卷积结果长度不超过 2^{23} ；递推阶数 $d < 2^{22}$ ，其他生成函数查询也须满足每次内部卷积的长度限制。卷积 $O(n \log n)$ ，已知递推的查询 $O(d \log(d+1) \log(i+1))$ ，RSPE 的推断另需 $O(sd)$ ， s 为样本数。有限样本不能证明某个递推对所有后续项成立，不要用于任意数列外推。本文件自带全部实现，不依赖 mint，也不要与其他定义 poly 的文件同时粘贴。

9.3 polynomial/mtt.cpp

非 NTT 模数下的多项式

什么时候用

目标模数运行时设置，三模 NTT 后 CRT 还原。卷积支持 2 至 INT_MAX 之间的任意模数，包括合数；常用 FPS 功能与 ntt-fast 对齐。

调用顺序与接口

- 先调用 `mint::setM(mod)`，再创建多项式。切换模数会清空 `fac/ifac/inv`，旧 `mint` 和 `poly` 值失效；同一时刻使用一个目标模数。
- `poly f={a0,a1,...}` 表示 $a_0+a_1x+\dots$ ，下标就是次数。
- `f*g` 是卷积；加减和数乘也支持。`Inv(f)` 返回模 x^n 的倒数， $n=f.size()$ ，`f[0]` 必须在目标模数下可逆。
- `Ln(f)` 要求 `f[0]=1`；`Exp(f)` 要求 `f[0]=0`；返回长度与输入相同。
- `diff/integ` 求导/积分，积分常数取零；`value(f,x)` 是单点求值。
- `BM(seq)` 求线性递推的分母多项式，`FSPE(F,G,k)` 求 F/G 的第 k 项，`RSPE(seq,k)` 从已有递推样本推第 k 项， k 从 0 开始。
- `Poly` 与 `poly` 是同一个类型；`lagrange(xs,ys)` 支持互异取值点，包括零点，返回次数小于点数的插值多项式。

最小示例（放入 main）：

```
mint::setM(1000000007);
poly f{1,1,0,0};
auto g=Inv(f); auto h=f*g; h.resize(4);
assert(h[0].x==1 && !h[1] && !h[2] && !h[3]);
auto e=Exp(Ln(f));
for(int i=0;i<4;i++)assert(e[i].x==f[i].x);
assert(FSPE({0,1},{1,-1,-1},10).x==55);
auto p=lagrange({0,1,2},{1,4,9});
assert(p[0].x==1 && p[1].x==2 && p[2].x==1);
```

使用边界

精度由输入 vector 长度决定，想求前 n 项就先 `resize(n)`，不要删掉需要保留的高位零。`operator/` 是相关运算 $(f/g)[i]=\sum_j f[i+j]*g[j]$ ，不是多项式除法。形式幂级数相除要 `f*Inv(g)` 后截断。各多项式文件均内置模数类型和全局表，只选一份，不要再粘贴 `ModInt`。卷积结果长度最多 2^{23} ，FPS 输入长度最多 2^{22} ，`Ln/Exp` 长度 n 还要小于模数，且 $1..n-1$ 均可逆。`init(n)` 要求 $n!$ 可逆，`integ(f)` 要求 $1..f.size()$ 可逆；使用质数模数且次数小于模数即可满足这些条件。`BM`、递推求值和插值中的除数也必须可逆；合数模数下仅仅非零或取值点互异不足以保证这一点。没有 `poly-new` 的 `Sqrt/Div/Eval`。`RSPE` 需要输入确实足以确定递推，通常至少给 2 倍阶数的样本，不适用于任意序列外推。

9.4 polynomial/ntt-fast.cpp

998244353 下的常用多项式

什么时候用

适用于固定模数下的大规模卷积与形式幂级数。卷积 $O(n \log n)$ ，BM 和普通插值为二次复杂度。

调用顺序与接口

- `poly f={a0,a1,...}` 表示 $a_0+a_1x+\dots$ ，下标就是次数。
- `f*g` 是卷积；加减和数乘也支持。`Inv(f)` 返回模 x^n 的倒数， $n=f.size()$ ，`f[0]` 必须非零。
- `Ln(f)` 要求 `f[0]=1`；`Exp(f)` 要求 `f[0]=0`；返回长度与输入相同。
- `diff/integ` 求导/积分，积分常数取零；`value(f,x)` 是单点求值。
- `BM(seq)` 求线性递推的分母多项式，`FSPE(F,G,k)` 求 F/G 的第 k 项，`RSPE(seq,k)` 从已有递推样本推第 k 项， k 从 0 开始。

最小示例（放入 main）：

```
poly f{1,1,0,0};
auto g=Inv(f); auto h=f*g; h.resize(4);
assert(h[0].x==1 && !h[1] && !h[2] && !h[3]);
auto e=Exp(Ln(f));
for(int i=0;i<4;i++)assert(e[i].x==f[i].x);
assert(FSPE({0,1},{1,-1,-1},10).x==55);
```

使用边界

精度由输入 vector 长度决定，想求前 n 项就先 `resize(n)`，不要删掉需要保留的高位零。`operator/` 是相关运算 $(f/g)[i]=\sum_j f[i+j]*g[j]$ ，不是多项式除法。形式幂级数相除要 `f*Inv(g)` 后截断。各多项式文件均内置模数类型和全局表，只选一份，不要再粘贴 `ModInt`。NTT 长度不超过 2^{23} ，`Inv/Ln/Exp` 的中间卷积也计入这个限制。`lagrange(x,y)` 要求取值点互异，允许零点。

9.5 polynomial/ntt-int64.cpp

ll 系数的旧版多项式

什么时候用

保留旧题适配接口，系数类型为 ll，默认 MOD=998244353。新题优先用 poly-fast 或 MTT。

调用顺序与接口

- poly 实际是 `vector<long long>`，输入系数应先归一化到 $[0, \text{MOD})$ 。
- `f*g` 为卷积，`inv/ln/exp/sqrt` 使用小写命名。
- `init(n)` 准备 `fac`、`ifac`、`Inv` 表；注意 `Inv` 是整数逆元数组，`inv` 才是幂级数求逆函数。

最小示例（放入 main）：

```
init(10); poly f{1,2},g{3,4}; auto h=f*g;
assert(h[0]==3 && h[1]==10 && h[2]==8);
poly a{1,1,0,0}; auto b=inv(a);
assert(b[0]==1 && b[1]==MOD-1);
```

使用边界

不要根据其他版本的接口大小写直接替换。旧版保留许多历史实现，本手册的可运行示例验证乘法和求逆；做递推推项优先选已核对接口的 MTT/poly-fast，避免依赖旧示例 solve。文件含 poly、LL、MP 等宏，组合时应留意污染。

9.6 polynomial/ntt.cpp

带长除、多点求值的多项式

什么时候用

同样固定模数 998244353，提供常用幂级数和 Sqrt、Div、Eval 等接口。

调用顺序与接口

- `poly f={a0,a1,...}` 表示 $a_0+a_1x+\dots$ ，下标就是次数。
- `f*g` 是卷积；加减和数乘也支持。`Inv(f)` 返回模 x^n 的倒数， $n=f.size()$ ，`f[0]` 必须非零。
- `Ln(f)` 要求 `f[0]=1`；`Exp(f)` 要求 `f[0]=0`；返回长度与输入相同。
- `diff/integ` 求导/积分，积分常数取零；`value(f,x)` 是单点求值。
- `BM(seq)` 求线性递推的分母多项式，`FSPE(F,G,k)` 求 F/G 的第 k 项，`RSPE(seq,k)` 从已有递推样本推第 k 项， k 从 0 开始。
- `Sqrt(f)` 当前仅接受 `f[0]=1`，取常数项为 1 的根。
- `Div(f,g,q,r)` 得到多项式长除的商与余式， g 最高次系数非零。`Eval(f,xs)` 返回各取值点的函数值。

最小示例（放入 main）：

```
poly f{1,1,0,0};
auto g=Inv(f); auto h=f*g; h.resize(4);
assert(h[0].x==1 && !h[1] && !h[2] && !h[3]);
auto e=Exp(Ln(f));
for(int i=0;i<4;i++)assert(e[i].x==f[i].x);
assert(FSPE({0,1},{1,-1,-1},10).x==55);
poly q,r; Div({1,2,1},{1,1},q,r);
assert(q.size()==2 && q[0].x==1 && q[1].x==1);
auto v=Eval({1,2,1},{0,1,2});
assert(v[0].x==1 && v[1].x==4 && v[2].x==9);
```

使用边界

精度由输入 vector 长度决定，想求前 n 项就先 `resize(n)`，不要删掉需要保留的高位零。`operator/` 是相关运算 $(f/g)[i]=\sum_j f[i+j]*g[j]$ ，不是多项式除法。形式幂级数相除要 `f*Inv(g)` 后截断。各多项式文件均内置模数类型和全局表，只选一份，不要再粘贴 `ModInt`。不支持任意常数项的模平方根。长除前清除除数尾部多余的零。`lagrange` 要求互异取值点，允许零点。NTT 上限为 2^{23} 。

10 计算几何 / geometry

10.1 geometry/geometry-2d.cpp

整数二维几何基础

什么时候用

点坐标和叉积均为 `ll`，提供凸包、面积与基本相交判断；需要另定义 `ld=long double`。

调用顺序与接口

- `Point(x,y)`：点向量加减；`a*b` 是叉积，`a^b` 是点积。
- `convex(points)` 求凸包，`convec_area(hull)` 返回两倍面积。
- `on_segment`、`strict_intersect`、`nonstrict_intersect` 分别判断在线段上、严格相交、含端点相交。

粘贴本文件之前：

```
using ld=long double;
```

最小示例（放入 `main`）：

```
vector<Point> a{{0,0},{2,0},{2,2},{0,2}};  
auto h=convex(a); assert(convec_area(h)==8);  
assert(on_segment(Point(1,0),Point(0,0),Point(2,0)));
```

使用边界

`convex` 自动排序去重。`min_cross` 要求顶点按凸包环序排列且没有共线中间点，支持旋转起点、顺时针， $O(\log n)$ 。射线包含端点，零方向按单点处理；退化三角形按三条边处理。坐标叉积、点积仍须保证不溢出 `ll`。`Point` 输出两个坐标，中间有空格；`output` 用于调试。